Cognizant GenC AEM Training

Final Evaluation — Complete Study Guide

Built from your handbook + your Educational Institute case study. Covers AEM 6.5 (Tags, CFs, OSGi, Sling Models, Components, HTL, XFs, Templates, Servlets, Workflows, Replication, MSM) plus Java fundamentals (OOP, Collections, Exceptions, Java 8) and Web fundamentals (HTML/CSS/JS, Servlets, JSP).

Field	Value
Project	GenC AEM Training - Educational Institute Site
AEM Version	6.5
Java Version	Java 11
Build Tool	Apache Maven
Total Topics	36 (from DCX-AEM Handbook Stage 2)
Prep Window	3–5 days (focused)

How to Use This Guide

This guide is split into four parts that match what you'll be asked about. Each topic follows a fixed structure so you can scan or deep-read.

Structure of each topic

- **Concept** — the textbook definition in plain English.
- **Why it exists** — the problem it solves.
- **How it works** — the mechanism, with code where useful.
- **In your project** — the green box ties it to your case study so you can speak from experience.
- **Interview tip** — the blue box gives you the exact thing to say if asked.
- **Watch out** — the yellow box flags gotchas evaluators often probe.

Suggested 5-day schedule

Day	Focus	Deliverable
Day 1	Part 1 — AEM Core (Components, HTL, Sling Models, OSGi)	Be able to draw the request flow on a whiteboard
Day 2	Part 2 — AEM Advanced (Servlets, Workflows, Replication, MSM, Assets, Tagging)	Explain MSM setup + write a servlet from memory
Day 3	Part 3 — Java fundamentals (OOP, Collections, Exceptions, Java 8)	Code 5 collection problems on paper
Day 4	Part 4 — Web (HTML/CSS/JS) + Servlets/JSP	Write a servlet+JSP form handler from scratch
Day 5	Mock interview — walk through case study end-to-end, drill weak spots	Confident 15-min project demo narration

PART 1 — AEM Core

This is the heart of every AEM evaluation. Roughly 60% of questions come from these topics because they're what you actually used to build the case study. Master these and you can recover from gaps elsewhere by anchoring answers in your project.

1. CMS / WCM Introduction & AEM Overview

Concept

A **Content Management System (CMS)** is software that lets non-technical users create, edit, organise, and publish digital content without writing code. A **Web Content Management System (WCM)** is a CMS specialised for websites — it adds templates, page hierarchies, preview, and publishing workflows.

Adobe Experience Manager (AEM) is Adobe's enterprise-grade WCM. It is built on the Java stack and uses a hierarchical JCR (Java Content Repository) instead of a relational database. AEM's two halves are **Sites** (web pages and components) and **Assets** (DAM — managing images, videos, PDFs, content fragments).

The three AEM environments

Environment	Purpose	Default Port
Author	Where authors create and edit content. Has the touch UI, dialogs, editor.	4502
Publisher	Public-facing read-only copy. Serves pages to end users.	4503
Dispatcher	Apache module that sits in front of the publisher. Caches HTML and load-balances. Adds security by hidin	(Apache)

AEM Tech Stack (bottom to top)

- **JCR (Apache Jackrabbit Oak)** — the content repository, stores everything as a tree of nodes and properties (JSR-283 spec).
- **Apache Sling** — REST-based web framework that maps URLs to JCR resources. The 'S' in HTL stands for Sling.
- **OSGi (Apache Felix)** — modular runtime. Every AEM feature is an OSGi bundle. Lets you install/start/stop modules without restarting AEM.
- **AEM-specific layer** — Sites, Assets, Forms, Communities, plus the authoring UI (Granite UI + Coral UI).

WYSIWYG

What You See Is What You Get — the authoring UI shows the page exactly as visitors will see it. Authors drag-and-drop components, click to edit, and see live preview without writing HTML.

■ **In your project:** Your Educational Institute site uses Author at `localhost:4502`. All 4 pages (Home, About Us, Courses, Contact Us) were authored there. You haven't deployed to a publisher, which is fine for a case study — but be ready to say *'in production we'd replicate to a publisher at 4503 fronted by a dispatcher'*.

■ **Interview Tip:** If asked *'What is AEM?'* answer in this order: (1) it's Adobe's enterprise WCM, (2) built on Java + Sling + OSGi + JCR, (3) split into Author/Publisher/Dispatcher, (4) used to manage websites and digital assets. Don't ramble — 30 seconds max.

2. AEM Local Setup & Key Interfaces

Local setup steps (typical)

- Install JDK 11 and set `JAVA_HOME`.
- Install Apache Maven and add its `bin` to PATH.
- Get the AEM 6.5 Quickstart JAR + license file from Adobe.
- Rename the JAR so the port is encoded: `aem-author-p4502.jar`.
- Run: `java -jar aem-author-p4502.jar`. First boot takes ~5 minutes (unpacks Quickstart).
- Browse `http://localhost:4502`, login as `admin / admin`.

Key interfaces you'll be asked about

Interface	URL	What it does
Sites Console	/sites.html	Browse and edit web pages.
Assets / DAM	/assets.html	Browse images, videos, content fragments.
CRXDE Lite	/crx/de/index.jsp	Low-level JCR browser — view raw nodes/properties. Like a file explorer fo
OSGi Console	/system/console	Manage bundles, components, configurations, services.
Package Manager	/crx/packmgr	Upload/download content packages (.zip).
Tools	/tools.html	Templates, tags, workflows, user admin, replication agents.

■ **Note:** Memorise these three URLs — evaluators love asking *'where would you go to check X?'*: `/system/console/bundles` (bundle state), `/system/console/configMgr` (OSGi configs), `/crx/de` (the repository).

■ **Interview Tip:** If asked *'how would you debug a Sling Model that isn't picking up changes?'* say: (1) check the bundle is Active at /system/console/bundles, (2) check the Sling Model is registered at /system/console/status-slingmodels, (3) tail error.log for AdaptionException.

3. Maven & AEM Project Structure

What is Maven?

Apache Maven is a Java build and dependency-management tool. It uses a `pom.xml` file (Project Object Model) that declares your project's dependencies, build plugins, and modules. Maven downloads dependencies from a central repository, compiles your code, runs tests, and packages the result.

AEM project = multi-module Maven project

Generated by the AEM Project Archetype (Adobe's official starter). Each module has a single responsibility:

Module	Output type	Purpose
core	JAR (OSGi bundle)	All Java code — Sling Models, OSGi services, Servlets, beans.
ui.apps	Content package (ZIP)	Components, clientlibs, templates → deploys to /apps/.
ui.apps.structure	(filter file)	Defines allowed paths in ui.apps — guards against accidental writes outside your
ui.config	Content package	OSGi configurations as .cfg.json — deploys to /apps/.../config.
ui.content	Content package	CF Models, sample pages, tags, XFs → deploys to /content/ and /conf/.
all	Container package	Bundles all the above into one deployable .zip.
it.tests	JAR (test)	Integration tests run against a running AEM.
ui.tests	JS (Cypress)	End-to-end UI tests.

The build command

```
mvn clean install -PautoInstallSinglePackage
```

- **clean** — deletes `target/` in every module.
- **install** — compiles, packages, and installs JARs/ZIPs into your local Maven repo (~/.m2).
- **-PautoInstallSinglePackage** — Maven profile that, after building, uploads the *all* package to AEM via its Package Manager HTTP API at localhost:4502.

filter.xml — the most-forgotten file

Each content-package module has a `META-INF/vault/filter.xml` that lists which JCR paths the package *owns*. When you deploy, the package overwrites those paths. Anything outside the filter is untouched.

```
<?xml version="1.0" encoding="UTF-8"?>
<workspaceFilter version="1.0">
  <filter root="/conf/genc-aem-training" mode="merge"/>
  <filter root="/content/genc-aem-training" mode="merge"/>
  <filter root="/content/dam/genc-aem-training" mode="merge"/>
  <filter root="/content/experience-fragments/genc-aem-training" mode="merge"/>
  <filter root="/content/cq:tags/genc-aem_training" mode="merge"/>
</workspaceFilter>
```

■ **Note: mode='merge'** means: keep existing child nodes, only add/update what's in the package. **Without it (mode='replace')**, redeploying your content package would wipe authored data every time. Always use merge for content paths.

■ **In your project:** Your ui.apps filter only owns `/apps/genc-aem-training/{clientlibs,components,i18n}`. Your ui.content owns `/conf/`, `/content/`, `/content/dam/`, `/content/experience-fragments/`, and

`/content/cq:tags/genc-aem_training`. Your interim issue — *'/conf not allowed in ui.apps'* — happened because /conf belongs in ui.content, not ui.apps.

■ **Interview Tip:** If asked *'why split into so many modules?'* say: **separation of concerns**. Code (immutable, /apps) is separated from content (mutable, /content). This lets you redeploy code without touching author data, and use different release cadences for each.

4. JCR Repository & Sling Resource Resolution

JCR — the content repository

Everything in AEM is stored in a single hierarchical tree of **nodes**, each with **properties**. There is no SQL database. The two top-level branches you care about:

- `/apps` — immutable. Your code (components, clientlibs, templates).
- `/content` — mutable. Authored content (pages, DAM assets, XFs, tags).
- `/conf` — configuration (templates, CF Models, context-aware configs).
- `/libs` — Adobe's code. Never modify; if you need to change something, **overlay** it under /apps with the same path.

Nodes have types

Node Type	Used for
cq:Page	A web page. Has a jcr:content child holding the actual data.
cq:Component	A component definition under /apps.
cq:Template	A page template.
dam:Asset	A DAM asset (image, content fragment).
cq:Tag	A tag.
nt:unstructured	Generic catch-all node. Most authored content.
nt:folder / sling:Folder	Folders.

Sling Resource Resolution — how URLs become content

When a request comes in, Sling resolves a URL to a JCR resource, then renders it with a script. Example URL: `/content/genc-aem-training/us/en/project/home-.html`

- Sling looks up the resource at that JCR path.
- The resource has a `sling:resourceType` property — e.g. `genc-aem-training/components/page`.
- Sling appends the URL's extension and looks under `/apps/genc-aem-training/components/page/` for `page.html`.
- That HTL file is executed to produce HTML.

URL structure

```
/content/path/to/page.selector1.selector2.html/suffix?query
```

- **Path** — JCR resource path.
- **Selectors** — dot-separated tokens (e.g. .mobile.json) used to vary rendering.
- **Extension** — picks the script (.html → HTL, .json → default JSON servlet).
- **Suffix** — extra path info passed to the script.
- **Query string** — standard HTTP query params.

■ **Interview Tip: Sling Resource Resolution flow** is a classic question. Memorise: *URL → JCR path → resource with sling:resourceType → script at /apps/{resourceType}/{name}.html → HTL execution →*

HTML response. Mentioning **selectors** and **extension-based script picking** impresses interviewers.

5. AEM Components — Anatomy & Custom Development

Concept

A **component** is the smallest reusable unit of authored content. It's a folder under `/apps/{project}/components/` containing all files needed to render and author one piece of UI (a heading, a card, a navigation, an entire header).

Component file structure

```
apps/genc-aem-training/components/coursecards/
■■■ .content.xml          # Component metadata (cq:Component node)
■■■ coursecards.html      # HTL template (rendering script)
■■■ _cq_dialog/
■   ■■■ .content.xml      # Authoring dialog (Granite UI)
■■■ _cq_editConfig.xml    # Edit toolbar config (optional)
■■■ _cq_template.xml      # Initial node structure when dropped (optional)
```

.content.xml — the component definition

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
          jcr:primaryType="cq:Component"
          jcr:title="Course Cards"
          jcr:description="Displays courses as cards with filter and limit."
          componentGroup="GenC AEM Training - Content"/>
```

- **jcr:primaryType="cq:Component"** — declares this folder as a component.
- **jcr:title** — name shown in the component browser.
- **componentGroup** — categorisation used by template policies. If blank, the component is hidden from authors.

Inheritance, Overlay, Override

Concept	Definition	Example
Inheritance	A component's sling:resourceSuperType points to another component. The child inherits its scripts and dialog.	Custom Title extends core/wcm/components/title/v3/title.
Overlay	You re-create the same JCR path under /apps that exists under /libs. Sling resolves /apps first, so your version wins.	Copy /libs/foundation/components/image to /apps/... and modify it.
Override	Within an inheritance chain: place a file in the child component with the same name as a file in the parent. The child version is used.	Override the parent's coursecards.html in your custom component.

Component Groups & Template Policies

Authors don't get every component on every page. Template policies define which components from which groups are allowed in each container. Setting your component's `componentGroup` is how it becomes discoverable.

■ **In your project:** Your project has 3 custom components: **Course Cards** (group: GenC AEM Training - Content) for the listing grid; **Site Header** and **Site Footer** (group: GenC AEM Training - Structure) for the XFs. The grouping separation lets template policies allow Header/Footer only in their template positions, while Course Cards goes in the main content area.

■ **Interview Tip:** If asked *'what's the difference between overlay and override?'*: overlay is cross-tree (/apps shadows /libs), override is within an inheritance chain (subclass shadows parent). They achieve the same goal — customizing without modifying the original — at different scopes.

6. Touch UI Dialogs (Granite UI)

Concept

A **dialog** is the form authors see when they click 'configure' on a component. It's defined declaratively in `_cq_dialog/.content.xml`. AEM 6.5 uses **Touch UI** (the modern responsive UI) built on **Granite UI** (the foundation widget library).

Dialog structure

```
_cq_dialog/.content.xml
■■■ content (granite/ui/components/coral/foundation/container)
    ■■■ items
        ■■■ tabs (granite/ui/components/coral/foundation/tabs)
            ■■■ items
                ■■■ tab1 (granite/ui/components/coral/foundation/container)
                ■   ■■■ items
                ■       ■■■ columns
                ■           ■■■ items
                ■               ■■■ field1 (textfield)
                ■               ■■■ field2 (pathfield)
                ■■■ tab2 ...
```

Common Granite UI field types

Field type	sling:resourceType	Used for
Text field	granite/ui/components/coral/foundation/form/textfield	Single-line text
Text area	granite/ui/components/coral/foundation/form/textarea	Multi-line text
Number field	granite/ui/components/coral/foundation/form/numberfield	Integers (min/max attrs)
Checkbox	granite/ui/components/coral/foundation/form/checkbox	Boolean
Select	granite/ui/components/coral/foundation/form/select	Dropdown
Pathfield	granite/ui/components/coral/foundation/form/pathfield	Pick a path from the repository (rootPath can constra
Tagfield	granite/ui/components/coral/foundation/form/tagfield	Pick tag(s) from a namespace
Multifield	granite/ui/components/coral/foundation/form/multifield	Variable-length list of items
RTE	cq/gui/components/authoring/dialog/richtext	Rich text editor

Multifield — the most-asked dialog widget

A multifield lets the author add N items (e.g. N navigation links). **Composite multifield** (composite="true") stores each item as a child node with multiple properties — necessary when each item has more than one field.

```
<navLinks jcr:primaryType="nt:unstructured"
   sling:resourceType="granite/ui/components/coral/foundation/form/multifield"
   composite="{Boolean}true"
   fieldLabel="Navigation Links">
   <field jcr:primaryType="nt:unstructured"
          sling:resourceType="granite/ui/components/coral/foundation/container"
          name="./navLinks">
      <items jcr:primaryType="nt:unstructured">
          <linkText jcr:primaryType="nt:unstructured"
                    sling:resourceType="granite/ui/components/coral/foundation/form/textfield"
                    fieldLabel="Link Text" name="./linkText" required="{Boolean}true"/>
          <linkURL jcr:primaryType="nt:unstructured"
                   sling:resourceType="granite/ui/components/coral/foundation/form/pathfield"
                   fieldLabel="Link URL" name="./linkURL"/>
```

```
        </items>
    </field>
</navLinks>
```

■ **Note:** Composite multifield stores each entry as a **child node** (item0, item1, item2, ...) under `./navLinks`, NOT as repeated properties. This means HTL must read them differently — see the HTL section.

Dialog values reach Sling Model via name attribute

Each field's `name='./headingText'` writes that value as a property on the component's JCR node. Your Sling Model reads it with `@ValueMapValue String headingText`. The field name and the Java field name must match (or use `@Named`).

■ **In your project:** Your Course Cards dialog has 3 fields: `./headingText` (textfield), `./limit` (numberfield 0-50), `./filterCategory` (tagfield rooted at `/content/cq:tags/genc-aem_training/course-category`). Header/Footer use composite multifields for nav links / footer links. Tagfield is rooted to constrain authors to your namespace.

Advanced dialog topics

- **Listeners** — JS code run on dialog events (loaded, beforesubmit). Attached via `cq:listeners` node.
- **Validators** — client-side checks declared with `validation` attribute.
- **Conditional sections** — show/hide fields with Granite UI's `granite:hide` / `granite:rel`.
- **Datasources** — populate select dropdowns dynamically (e.g. with a Java servlet returning a list).

■ **Interview Tip:** If asked *'how do you populate a dropdown from data in the JCR?'* answer: use a **datasource** — point the select field's `datasource` property to a sling:resourceType that resolves to a servlet, which writes a `DataSource` object into request attributes. The select reads it to populate options.

7. HTL (Sightly) — Templating Language

Concept

HTL (HTML Template Language), formerly called Sightly, is AEM's server-side templating language. It replaces JSP (which AEM still supports but discourages). HTL files use the `.html` extension and look like normal HTML — directives are added as `data-sly-*` attributes.

Why HTL over JSP

- **Secure by default** — automatically escapes output for the right context (HTML, attribute, JS, CSS, URL).
- **No Java code in templates** — forces separation of logic and presentation.
- **Designers can read it** — it's just HTML with extra attributes.
- **Faster** — pre-compiled to Java bytecode by the HTL compiler.

Core HTL block statements

Directive	Purpose
data-sly-use.x="ClassOrPath"	Instantiate a Sling Model or load a child resource as variable x.
data-sly-test="${expr}"	Render this element only if expr is truthy.
data-sly-list.item="${collection}"	Iterate the collection, expose each entry as item.
data-sly-repeat.item="${collection}"	Same as list but repeats the host element itself (no wrapper).
data-sly-include="path.html"	Include another HTL file.
data-sly-resource="${'path' @ resourceType='x'}"	Render a JCR resource through a different resource type.
data-sly-call="${template @ params}"	Call a defined template (like a function).
data-sly-template.name="${@ params}"	Define a reusable template.
data-sly-unwrap	Render only the contents, drop the host element.
data-sly-attribute="${map}"	Set HTML attributes from a map.
data-sly-element="${'h1'}"	Set the host element's tag name dynamically.
data-sly-set.x="${expr}"	Assign expr to variable x.

Expression language

```
${properties.headingText}        <!-- Read a JCR property -->
${model.courses}                 <!-- Call a Sling Model getter -->
${'Hello ' + properties.name}    <!-- String concatenation -->
${count > 5}                     <!-- Comparison -->
${a && b}                        <!-- Logical AND -->
${flag ? 'yes' : 'no'}           <!-- Ternary -->
${url @ context='uri'}            <!-- Context for safe output -->
${html @ context='html'}          <!-- Render trusted HTML -->
${date @ format='dd MMM yyyy'}    <!-- Format directive -->
```

Display contexts (XSS protection)

Context	Used for	What it escapes
text (default)	Plain text content	HTML tags become entities
html	Trusted HTML	Allows safe HTML tags, strips dangerous ones
attribute	HTML attribute value	Quote-safe encoding
uri	href, src	URL encoding, blocks javascript: scheme
scriptString	Inside a JS string	JS string encoding
styleString	Inside CSS	CSS encoding
number	Numeric value	Validates it's a number
unsafe	NO escaping	Avoid unless you really know.

Reading a composite multifield — the gotcha

Composite multifield items are child nodes, so HTL must load the resource first, then iterate its `.children`:

```
<nav data-sly-list.link="${navItems.children}"
     data-sly-use.navItems="navLinks">
   <a href="${link.linkURL}.html">${link.linkText}</a>
</nav>
```

The argument to `data-sly-use` can be either (1) a fully-qualified Java class name (Sling Model), or (2) the name of a child resource. Here `'navLinks'` is the child node name and `navItems` becomes a Resource object whose `.children` exposes item0, item1, etc.

■ **In your project:** Your `header.html` uses exactly this pattern: `data-sly-use.navItems='navLinks' + data-sly-list.link='${navItems.children}'`. Without `_cq_template.xml` pre-creating the child node, you get *SightlyException: No use provider could resolve identifier 'footerLinks'* — which is exactly the bug you hit and fixed.

HTL JavaScript Use-API (briefly)

HTL also supports JS files as logic helpers — alternative to Java Sling Models for simple cases. Less common in production but you should know it exists:

```
// helper.js
use(function() {
    return { greeting: "Hello " + properties.get("name") };
});
```

```
<!-- in HTL -->
<p data-sly-use.helper="helper.js">${helper.greeting}</p>
```

■ **Note:** HTL is strict — these are common parser errors: (1) **No XML entities** in expressions, e.g. `${a > b}` won't parse; use a Sling Model getter for comparisons. (2) **No method calls with arguments**, e.g. `${resource.getChild('x')}` fails — use `data-sly-use` to load the child.

■ **Interview Tip:** *'Why HTL over JSP?'* Three answers: secure by default (auto-escapes), strict separation of logic and view (no scriptlets), and faster execution (pre-compiled). If pushed: JSP is legacy AEM; HTL has been the recommended templating since AEM 6.0.

8. Sling Models — Bridging JCR and HTL

Concept

A **Sling Model** is a plain Java class with annotations that **adapts** a JCR Resource or HTTP Request into a clean Java object. It's the recommended way to put business logic behind a component. Sling Models replaced the older **WCMUse** and JSP scriptlets.

Why Sling Models

- Dependency injection for OSGi services, resource properties, request parameters.
- Testable — POJO with annotations, no AEM API required for unit tests.
- Auto-registered — no XML wiring, the @Model annotation is enough.
- Type-safe access to dialog values via @ValueMapValue.

Minimum viable Sling Model

```
package com.genc.core.core.models;

import org.apache.sling.api.SlingHttpServletRequest;
import org.apache.sling.api.resource.Resource;
import org.apache.sling.models.annotations.Model;
import org.apache.sling.models.annotations.DefaultInjectionStrategy;
import org.apache.sling.models.annotations.injectorspecific.ValueMapValue;
import org.apache.sling.models.annotations.injectorspecific.OSGiService;
import org.apache.sling.models.annotations.injectorspecific.Self;
import javax.annotation.PostConstruct;

@Model(
    adaptables = { SlingHttpServletRequest.class, Resource.class },
    defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL
)
public class CourseListModel {

    @Self
    private SlingHttpServletRequest request;

    @ValueMapValue
    private Long limit;

    @ValueMapValue
    private String filterCategory;

    @ValueMapValue
    private String headingText;

    @OSGiService
    private CourseConfigService courseConfigService;

    private List<CourseBean> courses;

    @PostConstruct
    protected void init() {
        // Logic: read fragments, filter, sort, limit
        // ...
    }

    public List<CourseBean> getCourses() { return courses; }
    public int getTotalCount() { return courses == null ? 0 : courses.size(); }
    public boolean isEmpty() { return getTotalCount() == 0; }
    public String getHeadingText() { return headingText; }
}
```

Key annotations

Annotation	Purpose
@Model	Declares the class as a Sling Model. adaptables = what types it can be created from.
@ValueMapValue	Inject a property from the resource's ValueMap (dialog value).
@OSGiService	Inject an OSGi service by interface type.
@Self	Inject the adaptable itself (the request or resource).
@SlingObject	Inject Sling-provided objects (ResourceResolver, Resource, etc.).
@Inject	Generic injection — finds any matching injector. Less specific than typed annotations.
@Named("propName")	Override the property name to inject.
@Default(values=...)	Default value if missing.
@PostConstruct	Method called after all fields are injected. Put init logic here.
@Optional / required	Control whether a missing value causes a failure.

DefaultInjectionStrategy

OPTIONAL (recommended) means any missing field is null/zero, not a failure. **REQUIRED** means the whole Sling Model fails to adapt if any field is missing — useful in strict contracts but brittle for AEM dialogs where fields are often empty.

Adaptables — Request vs Resource

- **Request** — can inject request parameters, selectors, suffixes, current page. Use when you need request context.
- **Resource** — only adapts from a JCR resource. No request context. Lighter, used in templates that loop resources.
- List both as in your CourseListModel to get the most flexibility.

■ **In your project:** Your CourseListModel reads three dialog fields (limit, filterCategory, headingText), injects the OSGi service (CourseConfigService), then in `@PostConstruct init()` it walks the DAM tree, identifies Course CFs by their `cq:model` property, builds a list of CourseBean, applies filter / sort / limit. The bean carries pre-formatted date strings so HTL doesn't have to deal with Calendar objects (which is one of your documented issues).

Calling a Sling Model from HTL

```
<div data-sly-use.model="com.genc.core.core.models.CourseListModel">
    <h2>${model.headingText}</h2>
    <p>Showing ${model.totalCount} courses</p>
    <article data-sly-list.course="${model.courses}">
        <h3>${course.title}</h3>
        <p>${course.description @ context='html'}</p>
    </article>
</div>
```

■ **Interview Tip:** If asked *'how does Sling Model know to inject a dialog value into @ValueMapValue?'*: AEM finds the resource adapted, gets its ValueMap (a Map<String,Object> of all properties), and copies properties whose names match the field name into the model. Mention that **field name == JCR property name** by default, override with @Named.

Sling Model registry — verification

After deploying, visit `/system/console/status-slingmodels`. Your model should appear in the list of registered classes. If not, the bundle is inactive or the package `com.genc.core.*` wasn't exported in the Maven bundle plugin config.

9. OSGi — Components, Services, Configuration

What is OSGi

OSGi (Open Services Gateway initiative) is a Java framework for building modular applications. Every feature in AEM is an **OSGi bundle** — a JAR with extra metadata. Bundles can be installed, started, stopped, updated, or uninstalled at runtime without restarting AEM.

Bundle vs JAR

Aspect	Plain JAR	OSGi Bundle
MANIFEST	Basic metadata	Has Bundle-SymbolicName, Bundle-Version, Import-Package, Expo
Visibility	All classes visible to classpath	Only exported packages are visible to others
Lifecycle	Loaded once at JVM start	INSTALLED → RESOLVED → STARTING → ACTIVE → STOPPIN
Dependencies	Classpath jars	Declared imports resolved by OSGi at runtime

OSGi Service — concept

An **OSGi Service** is a Java interface implemented by a class registered in the OSGi service registry. Other components can declare a dependency on the interface and OSGi injects the matching implementation. This is dependency injection at the bundle level.

The 4-file pattern (from your project)

File	Purpose
CourseConfig.java	@ObjectClassDefinition — config schema (an annotation-as-interface)
CourseConfigService.java	Plain Java interface — the contract exposed to consumers
CourseConfigServiceImpl.java	@Component class — implements the interface, reads config
*.cfg.json	Configuration values, deployed as JSON in ui.config

1. Config interface (@ObjectClassDefinition)

```
@ObjectClassDefinition(
    name = "GenC AEM Training - Course Configuration",
    description = "Configuration for the Educational Institute course listing"
)
public @interface CourseConfig {
    @AttributeDefinition(name = "Course Root Path",
        description = "DAM root path where course CFs are stored")
    String courseRootPath() default "/content/dam/genc-aem-training/courses";

    @AttributeDefinition(name = "Default Limit",
        description = "Default number of courses to display")
    int defaultLimit() default 10;
}
```

2. Service interface (public API)

```
public interface CourseConfigService {
    String getCourseRootPath();
    int getDefaultLimit();
}
```

3. Implementation class (@Component + @Designate)

```
@Component(service = CourseConfigService.class, immediate = true)
@Designate(ocd = CourseConfig.class)
public class CourseConfigServiceImpl implements CourseConfigService {

    private String courseRootPath;
    private int defaultLimit;

    @Activate
    protected void activate(CourseConfig config) {
        this.courseRootPath = config.courseRootPath();
        this.defaultLimit  = config.defaultLimit();
    }

    @Override public String getCourseRootPath() { return courseRootPath; }
    @Override public int getDefaultLimit()      { return defaultLimit; }
}
```

4. Deployed config (.cfg.json in ui.config)

```
// Path: ui.config/.../osgiconfig/config/
//   com.genc.core.core.services.impl.CourseConfigServiceImpl.cfg.json
{
  "courseRootPath": "/content/dam/genc-aem-training/courses",
  "defaultLimit": 10
}
```

OSGi Component lifecycle annotations

Annotation	When called	Use
@Activate	Component activated (config bound, deps satisfied)	Init / read config
@Modified	Config values changed at runtime	Re-read config without restart
@Deactivate	Component shutting down	Cleanup, close resources
@Reference	Field/method — inject another OSGi service	Service dependency

Service properties (advanced)

```
@Component(
    service = MyService.class,
    property = {
        "service.ranking:Integer=100",     // higher wins if multiple impls
        "process.label=Approval"           // arbitrary metadata
    }
)
```

Config types — Sling resolution order

AEM resolves OSGi configs in this order (later overrides earlier): (1) Default in @AttributeDefinition, (2) `.cfg.json` in /apps/ from your code, (3) Web Console changes (stored in JCR), (4) Runmode-specific configs like `config.author` or `config.publish.prod`.

■ **Note: service vs servicefactory**: a regular `@Component` service is a singleton — one instance shared by all consumers. A **ServiceFactory** creates a new instance per consuming bundle. ServiceFactory is rare; the default singleton is almost always what you want.

■ **In your project:** Your CourseConfigService is a textbook example: one config (DAM root path + default limit), injected into CourseListModel via `@OSGiService`. The dialog `limit` field overrides the OSGi default — that's the configurability story to tell in your demo: *'authors can override per-component, admins can change the default for all components without redeploying.'*

■ **Interview Tip:** Top OSGi question: *'What's the difference between an OSGi config and a Sling Model @ValueMapValue?'* OSGi config is global, set by admins via Web Console or .cfg.json, applies to all instances. @ValueMapValue reads per-component-instance values from dialog. Use OSGi for environment-level settings; use dialog for content-level settings.

10. Editable Templates

Static vs Editable templates

Aspect	Static Template	Editable Template
Location	/apps/...	/conf/.../settings/wcm/templates/
Created by	Developer (XML)	Template Author (UI)
Structure changes	Require code redeploy	Live edits propagate to pages
Policies	Hard-coded design dialog	Centralised policies per container
When to use	Legacy projects	All new development (since AEM 6.2)

Three sections of an editable template

- **Structure** — locked-down layout: header, footer, content area. Defined by template authors.
- **Initial Content** — what new pages start with (a hero image, a default title).
- **Layout** — responsive breakpoints (mobile, tablet, desktop).

Template Policies

A policy is a configuration applied to a component in the template's structure. It defines (a) which child components are allowed, (b) default settings. Policies are stored under `/conf/{project}/settings/wcm/policies/` and referenced from the template structure.

Template lifecycle

Status	Authors can use it?	When
Draft	No	Being built
Enabled	Yes	Production-ready
Disabled	No (existing pages still work)	Deprecated

■ **In your project:** Your **Content Page** template at `/conf/genc-aem-training/settings/wcm/templates/content-page` has: top = Experience Fragment placeholder pointing to Header XF; middle = Layout Container with policy allowing Course Cards + core components + Content Fragment List etc.; bottom = XF placeholder for Footer. All 4 pages (Home, About Us, Courses, Contact) use this template — that's why headers/footers are consistent.

■ **Interview Tip:** *'Why editable templates over static?'* Three reasons: (1) template authors (not just developers) can change structure, (2) policy changes propagate to all pages instantly without redeploy, (3) Adobe deprecated static templates — all new projects use editable.

11. Client Libraries (Clientlibs)

Concept

Client Libraries are AEM's way of bundling and delivering CSS, JavaScript, and other static assets to the browser. A clientlib is a JCR node of type `cq:ClientLibraryFolder` with a category and lists of files.

Structure

```
clientlib-site/
■■■ .content.xml           # categories=[genc-aem-training.site]
■■■ css.txt                # manifest of CSS files in load order
■■■ js.txt                 # manifest of JS files
■■■ css/
■   ■■■ coursecards.css
■   ■■■ header-footer.css
■■■ js/
    ■■■ (none)
```

.content.xml

```
<jcr:root xmlns:cq="http://www.day.com/jcr/cq/1.0"
          xmlns:jcr="http://www.jcp.org/jcr/1.0"
          jcr:primaryType="cq:ClientLibraryFolder"
          allowProxy="{Boolean}true"
          categories="[genc-aem-training.site]"/>
```

- **categories** — names used to opt-in. Pages reference categories, not paths.
- **allowProxy** — exposes clientlibs at /etc.clientlibs/* so they're not served from /apps (which would expose code paths).

css.txt / js.txt — the manifest

```
# Comments start with #
#base=css
coursecards.css
header-footer.css
```

Loading clientlibs in HTL

```
<!-- In head: CSS only -->
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html"
     data-sly-call="${clientlib.css @ categories='genc-aem-training.site'}"/>

<!-- In body footer: JS only -->
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html"
     data-sly-call="${clientlib.js @ categories='genc-aem-training.site'}"/>

<!-- Both at once -->
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html"
     data-sly-call="${clientlib.all @ categories='genc-aem-training.site'}"/>
```

dependencies vs embed

Property	Behavior	When to use
dependencies	Listed clientlibs are loaded BEFORE this one, as separate files	Shared libs like jQuery — each page links it once
embed	Listed clientlibs are MERGED INTO this one — single combined file	Combine many small libs to reduce HTTP requests

Production optimisations

- **Minify** — strips whitespace and comments (enabled in OSGi: HTML Library Manager).
- **Gzip** — applied by Dispatcher / Apache.

- **Debug mode** — append `?debugClientLibs=true` to URL to see individual files instead of merged.

■ **In your project:** Your **clientlib-site** has category `genc-aem-training.site` and contains `coursecards.css` + `header-footer.css`. No JS files. The page template loads this category from its head section.

■ **Interview Tip:** If asked *'how do you serve CSS in AEM?'*: clientlibs with categories. Mention **allowProxy** for security (/etc.clientlibs instead of /apps paths), **embed** for merging libraries, and **?debugClientLibs=true** for debugging.

12. Tags & Tagging Framework

Concept

Tags are AEM's hierarchical taxonomy system. They let you categorise content (pages, assets, fragments) using a controlled vocabulary. Tags live under `/content/cq:tags/` as `cq:Tag` nodes.

Tag identifier format

```
namespace:parent/child/grandchild
```

Example: `genc-aem_training:course-category/engineering`. The part before the colon is the **namespace** (top-level folder under `/content/cq:tags/`); after the colon is the path within.

Why namespaces matter

- Isolation — different projects, languages, business units don't collide.
- Scoping — pickers (tagfield) can be rooted at a namespace.
- Cleanup — uninstalling a project can remove a single namespace.

■ **Note:** AEM auto-converts hyphens to underscores in namespace node names. You intended `genc-aem-training`, the actual node became `genc-aem_training`. All references (CF Model field root path, Sling Model code, dialog configuration) must use the underscore variant — this is the bug documented in your case study.

Tagging API in Java

```
// Get the TagManager
ResourceResolver resolver = request.getResourceResolver();
TagManager tagManager = resolver.adaptTo(TagManager.class);

// Resolve a tag ID to a Tag object
Tag tag = tagManager.resolve("genc-aem_training:course-category/engineering");
if (tag != null) {
    String title = tag.getTitle();
    String desc  = tag.getDescription();
    String path  = tag.getPath();
}

// Find all tags applied to a resource
Tag[] tags = tagManager.getTags(resource);

// Apply tags to a resource
tagManager.setTags(resource, new Tag[]{ tag1, tag2 });
```

Tag search

```
// Find resources tagged with this tag
RangeIterator<Resource> results = tagManager.find(
    "/content/genc-aem-training",
    new String[]{ "genc-aem_training:course-category/engineering" },
    true
);
```

■ **In your project:** Your project's tag setup: namespace `genc-aem_training` with parent `course-category` and 10 child tags (engineering, management, arts, science, commerce, medical, law, it, design, education). The Course CF Model's courseCategory field is rooted at `/content/cq:tags/genc-aem_training/course-category` so authors only see those 10 options. Your CourseListModel resolves tags via `tagManager.resolve()` with a string-extraction fallback (if TagManager returns null, parse the last path segment and capitalize).

■ **Interview Tip:** If asked *'how would you scope a tag picker?'*: set the tagfield's `rootPath` property in the dialog to the namespace folder. If asked *'what's the difference between a tag and a category?'*: in

AEM, tags ARE the implementation of categories — same concept, AEM just calls them tags.

13. Content Fragments — Headless Content

Concept

A **Content Fragment (CF)** is structured, presentation-free content. Think of it as *'data without HTML'* — pure fields like title, description, dates, references. CFs are stored in the DAM as `dam:Asset` nodes.

Content Fragment Model (the schema)

A **CF Model** defines the structure: which fields, what types, validation. Like a Java class. Lives at `/conf/{project}/settings/dam/cfm/models/`. From one model, authors create many fragments — like objects of a class.

Field types in a CF Model

Type	Stores	When to use
Single-line Text	String	Titles, names
Multi-line Text	String (HTML if Rich Text)	Descriptions, body content
Number	Long/Double	Counts, prices
Boolean	true/false	Flags
Date and Time	Calendar	Dates
Enumeration	String	Predefined options (dropdown)
Tags	String (tag ID) or [String]	Categorisation
Content Reference	String (path)	Link to another asset (image)
Fragment Reference	String (path)	Link to another CF (nested content)
JSON Object	String	Complex structured data

CF vs DAM Asset vs Image

- An **image in DAM** is a binary asset with metadata.
- A **Content Fragment** is a dam:Asset whose binary is a JSON payload of the fields.
- A CF carries a special property `cq:model` = path to its model.

Reading CFs in Java

```
// Adapt a resource to ContentFragment
Resource cfResource = resourceResolver.getResource("/content/dam/.../bca");
ContentFragment cf = cfResource.adaptTo(ContentFragment.class);
if (cf != null) {
    String title = cf.getElement("courseTitle").getValue().getValue(String.class);
    Calendar startDate = cf.getElement("startDate").getValue().getValue(Calendar.class);
    String[] tags = cf.getElement("courseCategory").getValue().getValue(String[].class);
}

// Or read directly from jcr:content/data/master
Resource data = cfResource.getChild("jcr:content/data/master");
ValueMap props = data.getValueMap();
String title = props.get("courseTitle", String.class);
```

CF Component (OOTB)

AEM ships a Content Fragment Component that authors drop on a page and configure to show a single fragment. Useful for simple cases. For listings or custom presentations (like your Course Cards) you build a Sling Model that reads multiple CFs.

■ **In your project:** Your **Course** CF Model has 6 fields: courseTitle, courseDescription (Rich Text), startDate, endDate, courseCategory (Tags rooted at course-category), courseImage (Content Reference). You created 12 fragments under `/content/dam/genc-aem-training/courses/`. Your CourseListModel walks the DAM, identifies Course CFs by `cq:model`, and renders them via Course Cards. This is the **headless pattern**: same data could be consumed by a mobile app via the GraphQL API.

GraphQL API (advanced)

AEM exposes CFs via GraphQL at `/content/cq:graphql/global/endpoint`. You define a persisted query, then fetch fragments from any external client. This makes CFs the foundation of AEM's **headless** story.

■ **Interview Tip:** *'What's the difference between a Content Fragment and an Experience Fragment?'* — a common trick question. CF = **data without presentation** (structured content for any channel). XF = **presentation with content baked in** (fully-authored UI block for embedding). CFs are headless; XFs are page-fragment reuse.

14. Experience Fragments (XFs)

Concept

An **Experience Fragment** is an authored block of page content — including its layout and styling — that you can embed on multiple pages or even export to other channels. Stored under `/content/experience-fragments/`. Authored like a page but never published as a standalone page.

Why XFs

- **Single source of truth** — change the header once, every page using it updates.
- **Cross-channel** — same XF rendered on web, mobile, AEM Screens, even pushed to Facebook/Pinterest.
- **Variations** — one XF can have multiple variations (web, mobile, email). Each is a sibling under the XF's parent.
- **Versioning** — XFs are versioned like pages — you can roll back.

XF structure in JCR

```
/content/experience-fragments/genc-aem-training/
■■■ site/
    ■■■ Header/
    ■   ■■■ master/          (the default variation, a cq:Page)
    ■   ■■■ mobile/          (optional alternative variation)
    ■■■ Footer/
        ■■■ master/
```

Embedding an XF on a page

Drop the **Experience Fragment** core component on a page and point it at the XF path. Or in a template's structure, place an XF placeholder so all pages using the template automatically include it (this is what your Content Page template does for header/footer).

XF vs CF (again — interviewers love this)

Aspect	Content Fragment	Experience Fragment
What it is	Structured data (fields)	Authored UI block (components + layout)
Storage	/content/dam/ as dam:Asset	/content/experience-fragments/ as cq:Page
Rendering	Needs a component to render	Has its own render — drop on a page
Primary use	Headless content for any channel	Reusable page sections (header, footer, hero)
Author UI	Form-based field editor	Page editor with drag-and-drop

■ **In your project:** You built two XFs: **Header** at `/content/experience-fragments/genc-aem-training/site/Header/master` and **Footer** at the parallel Footer path. Each XF contains your custom Site Header / Site Footer component (with the multifield navigation links). The Content Page template's top and bottom XF placeholders point at these. Result: all 4 pages share consistent header/footer, change-once-update-everywhere.

■ **Interview Tip:** Strong answer to *'why use XFs for header/footer?'*: (1) single source of truth, (2) editable by content authors without code changes, (3) supports variations for different devices/channels, (4) versioned so rollback is easy. Contrast with hardcoding in template: *changes need a developer redeploy*.

15. Site Pages & Page Component

What is a page

A **page** in AEM is a JCR node of type `cq:Page` with a `jcr:content` child that holds the actual content. The jcr:content node has a `sling:resourceType` pointing to a **page component**, which is the top-level HTL that renders the entire page.

```
cq:Page  (the page)
■■■ jcr:content  (the content node)
    ■■■ @sling:resourceType = "genc-aem-training/components/page"
    ■■■ @cq:template = "/conf/.../templates/content-page"
    ■■■ @jcr:title = "Home"
    ■■■ (child nodes for components dropped on the page)
        ■■■ header  (XF embed)
        ■■■ root    (responsivegrid / Layout Container)
        ■   ■■■ title
        ■   ■■■ coursecards
        ■   ■■■ ...
        ■■■ footer  (XF embed)
```

Page component anatomy

A page component lives under `/apps/{project}/components/page/`. Its `page.html` typically: (1) renders <head> with clientlibs, (2) renders the XF for the header, (3) renders the responsive grid for content, (4) renders the XF for the footer.

URL pattern

```
/content/{project}/{country}/{lang}/{section}/{page}.html
```

AEM convention is geographic + language path: `/content/genc-aem-training/us/en/project/home-`. The `.html` extension triggers HTL rendering.

Page properties

Every page has properties accessible via Page Information → Page Properties. Common ones: page title, navigation title, tags, on/off times, SEO description, social media metadata. Stored on jcr:content.

■ **In your project:** Your 4 pages: Home (`home-`), About Us, Courses (`courses-`), Contact Us — all under `/content/genc-aem-training/us/en/project/`, all using the Content Page template. Trailing hyphens are an authoring quirk; the actual URLs work fine. Be ready to navigate this in your demo via the sites console.

■ **Interview Tip:** If asked *'what's the difference between a page and jcr:content?'*: cq:Page is the container with metadata (created/modified dates, status); jcr:content is the actual editable node where all components and properties live. Authoring a page means editing jcr:content.

PART 2 — AEM Advanced

These topics aren't in your case study but are in the handbook — expect at least 2-3 questions from this section. The most common: Sling Servlets, Workflows, Replication, MSM.

16. Sling Servlets

Concept

A **Sling Servlet** is a Java class extending `SlingSafeMethodsServlet` (read-only) or `SlingAllMethodsServlet` (read + write), registered as an OSGi component, used to handle HTTP requests. Servlets are how you build custom endpoints (JSON APIs, form processors, datasources) in AEM.

Two ways to register a servlet

Style	When triggered	Example
By path	When request URL matches the exact path	/bin/genc/courses.json — direct hit
By resource type	When a resource with that sling:resourceType is requested	A resource of type genc/components/api

Servlet registered by path

```
package com.genc.core.core.servlets;

import org.osgi.service.component.annotations.Component;
import org.apache.sling.api.servlets.SlingSafeMethodsServlet;
import org.apache.sling.api.SlingHttpServletRequest;
import org.apache.sling.api.SlingHttpServletResponse;

import javax.servlet.Servlet;
import java.io.IOException;

@Component(
    service = Servlet.class,
    property = {
        "sling.servlet.paths=/bin/genc/courses",
        "sling.servlet.methods=GET",
        "sling.servlet.extensions=json"
    }
)
public class CourseListServlet extends SlingSafeMethodsServlet {

    @Override
    protected void doGet(SlingHttpServletRequest req, SlingHttpServletResponse res)
            throws IOException {
        res.setContentType("application/json");
        res.getWriter().write("{\"courses\": []}");
    }
}
```

Servlet registered by resource type (preferred)

```
@Component(
    service = Servlet.class,
    property = {
        "sling.servlet.resourceTypes=genc-aem-training/components/api/courses",
        "sling.servlet.methods=GET",
        "sling.servlet.extensions=json",
        "sling.servlet.selectors=list"
    }
)
public class CourseListServlet extends SlingSafeMethodsServlet { ... }
```

Path vs Resource Type — which to use?

By path	By resource type
Quick, like a traditional servlet endpoint	Sling-idiomatic; URL stays content-driven
No JCR resource needed	Requires a resource at the URL
Path is hardcoded; not portable	Many resources can share one servlet
Discouraged for production	Recommended best practice
Use for utility endpoints (/bin/...)	Use for component endpoints

Required servlet properties

- **sling.servlet.paths** — exact path(s) to bind to.
- **sling.servlet.resourceTypes** — sling:resourceType(s) to bind to.
- **sling.servlet.methods** — HTTP methods (GET, POST, etc.). Default is GET.
- **sling.servlet.extensions** — URL extension filter (json, html).
- **sling.servlet.selectors** — URL selector filter (.list., .detail.).

Default servlet & Sling servlet resolver

When no custom servlet matches, Sling falls back to the **DefaultGetServlet** which renders JCR resources as HTML/JSON/XML based on the extension. The **SlingServletResolver** handles the lookup — visible at `/system/console/servletresolver` where you can input a URL and see which servlet would handle it.

■ **Note: Servlet lifecycle**: same as any OSGi component — INSTALLED → RESOLVED → ACTIVE. The servlet class itself doesn't have init/destroy in the JEE sense; use @Activate / @Deactivate on the OSGi component.

■ **Interview Tip:** *'Path vs resource type registration — which is better and why?'* Resource type. Reasoning: (1) URL stays content-driven (a JCR resource is the endpoint), (2) multiple resources can share one servlet, (3) Adobe explicitly recommends it. Paths are fine for /bin/* utility endpoints but discouraged otherwise.

17. Users, Groups & Permissions (RBAC)

Concept

AEM uses **Role-Based Access Control**: **Users** belong to **Groups**, and **Access Control Lists (ACLs)** grant/deny **privileges** on JCR paths. Permissions inherit downward (a permission on /content applies to all child pages).

Default groups in AEM

Group	What they can do
administrators	Full access to everything
content-authors	Create/edit pages, assets
dam-users	Work with DAM assets
workflow-users	Participate in workflows
contributors	Limited content edit
everyone	Implicit group — every authenticated user is a member

JCR privileges (the building blocks)

Privilege	Grants
jcr:read	View nodes and properties
jcr:modifyProperties	Edit property values on existing nodes
jcr:addChildNodes	Create child nodes
jcr:removeNode	Delete a node
jcr:removeChildNodes	Delete child nodes
jcr:readAccessControl	View ACLs
jcr:modifyAccessControl	Change ACLs
jcr:all	AGGREGATE — all JCR privileges combined
crx:replicate	AEM-specific: publish to publisher. NOT included in jcr:all.

■ **Note: jcr:all does NOT include crx:replicate**. crx:replicate is an AEM-specific privilege outside the JCR core spec. For authors who need to publish, you must grant both. This is a classic interview gotcha.

Access Control Entry (ACE) — concrete example

On `/content/genc-aem-training` for group `gencinstitute-authors`: ALLOW jcr:read, jcr:modifyProperties, jcr:addChildNodes, jcr:removeNode, crx:replicate. This grants full author rights on the site tree.

Permission inheritance & order

- Permissions flow downward — granted on a parent path apply to all descendants unless overridden.
- Order matters: DENY at a deeper node overrides ALLOW at a parent.
- If a user is in multiple groups with conflicting permissions, DENY wins.

Impersonate functionality

AEM lets one user temporarily act as another (for support/debugging). Configured under Tools → Security → Users → User Properties → Impersonators. When impersonating, audit logs record both the actual user and the impersonated identity.

■ **In your project:** Your RBAC setup has 3 groups: **gencinstitute-authors** (regular content authors, full rights on /content and /content/dam but explicitly DENY write on /content/experience-fragments — prevents accidental header/footer changes); **gencinstitute-xf-admins** (full rights on the XF path, can edit Header/Footer XFs); **gencinstitute-readers** (jcr:read only). Each group has one demo user (author1, admin-xf1, reader1). This demonstrates **separation of duties** and **least privilege**.

■ **Interview Tip:** Top question: *'How do you publish-only one role, not another?'* Grant crx:replicate to the publishers' group on the relevant path. Mention that crx:replicate is AEM-specific and not part of jcr:all — score bonus points.

18. Service Users, Logging & Runmodes

Service User

OSGi services should NOT use the admin session — security risk. Instead, you create a **Service User** (a user with no password, used only by code) and a **Service User Mapping** that binds your bundle's subservice name to that user.

Creating a service user (Repoinit)

```
# Repoinit script in OSGi config:
# org.apache.sling.jcr.repoinit.RepositoryInitializer-genc.cfg.json
{
  "scripts": [
    "create service user genc-course-reader",
    "set ACL for genc-course-reader",
    "  allow jcr:read on /content/dam/genc-aem-training",
    "end"
  ]
}
```

Service User Mapping (OSGi config)

```
// org.apache.sling.serviceusermapping.impl.ServiceUserMapperImpl.amended-genc.cfg.json
{
  "user.mapping": [
    "com.genc.core:course-reader=[genc-course-reader]"
  ]
}
```

Using it from Java

```
@Reference
private ResourceResolverFactory resolverFactory;

private ResourceResolver getServiceResolver() throws LoginException {
    Map<String, Object> params = Collections.singletonMap(
        ResourceResolverFactory.SUBSERVICE, "course-reader"
    );
    return resolverFactory.getServiceResourceResolver(params);
}

public void readSomething() {
    try (ResourceResolver resolver = getServiceResolver()) {
        Resource r = resolver.getResource("/content/dam/...");
        // ...
    } catch (LoginException e) {
        log.error("Service resolver failed", e);
    }
}
```

■ **Note: Never use resolverFactory.getAdministrativeResourceResolver()** — deprecated, removed in newer Sling versions. Always go through service user mapping.

Logging — SLF4J

AEM uses **SLF4J** with **Logback**. Get a logger per class, log at appropriate levels.

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger LOG = LoggerFactory.getLogger(CourseListModel.class);

LOG.debug("Reading courses from {}", courseRootPath);
LOG.info("Loaded {} courses", count);
LOG.warn("Tag {} could not be resolved", category);
LOG.error("Failed to read fragment {}", path, exception);  // last arg is the exception
```

Custom logger configuration (OSGi)

```
// org.apache.sling.commons.log.LogManager.factory.config-genc.cfg.json
{
  "org.apache.sling.commons.log.names": ["com.genc.core"],
  "org.apache.sling.commons.log.level":  "DEBUG",
  "org.apache.sling.commons.log.file":   "logs/genc.log",
  "org.apache.sling.commons.log.pattern": "{0,date,dd.MM.yyyy HH:mm:ss.SSS} *{4}* [{2}] {3} {5}"
}
```

Log file locations

File	What it contains
crx-quickstart/logs/error.log	All errors and warnings; default destination
crx-quickstart/logs/request.log	Every HTTP request with timing
crx-quickstart/logs/access.log	Apache-style access log
crx-quickstart/logs/upgrade.log	AEM upgrade messages
crx-quickstart/logs/stderr.log / stdout.log	Process stderr/stdout

Runmodes

A **runmode** is a label applied to an AEM instance: **author** or **publish** are built-in, you can add custom ones like **dev**, **stage**, **prod**. Runmodes are set via JVM arg: `-Dsling.run.modes=author,dev`.

Runmode-specific OSGi configs

```
ui.config/.../osgiconfig/
■■■ config/                          # Always applied
■■■ config.author/                   # Only on author instances
■■■ config.publish/                  # Only on publishers
■■■ config.author.dev/               # Author + dev runmode
■■■ config.author.prod/              # Author + prod runmode
■■■ config.publish.prod/             # Publisher + prod runmode
```

■ **Interview Tip:** If asked *'how do you configure different DB endpoints for dev vs prod?'*: put the OSGi config in runmode-specific folders. AEM picks the most specific match. Mention `config.author.prod` is more specific than `config.author` which is more specific than `config`.

19. AEM Assets (DAM)

Concept

DAM (Digital Asset Manager) is AEM's media library. Stores images, videos, PDFs, Content Fragments. Everything in `/content/dam/` is a `dam:Asset` node.

Asset node structure

```
mybanner.jpg  (dam:Asset)
■■■ jcr:content  (dam:AssetContent)
    ■■■ @jcr:title = "My Banner"
    ■■■ metadata/  (extracted EXIF, IPTC, custom fields)
    ■■■ renditions/
        ■■■ original  (the uploaded binary)
        ■■■ cq5dam.thumbnail.48.48.png
        ■■■ cq5dam.thumbnail.140.100.png
        ■■■ cq5dam.thumbnail.319.319.png
```

Asset processing workflow

When you upload an asset, AEM runs the **DAM Update Asset** workflow which: (1) extracts metadata, (2) generates thumbnails, (3) creates web-optimized renditions, (4) runs subasset extraction (e.g., extracts pages from a PDF), (5) tags the asset.

Asset metadata

- **EXIF** — camera data extracted from images (ISO, focal length, dimensions).
- **IPTC** — editorial metadata (title, copyright, keywords).
- **Custom metadata schemas** — define your own fields per folder (Tools → Assets → Metadata Schemas).

Assets API

```
// Read an asset
Resource assetResource = resolver.getResource("/content/dam/.../image.jpg");
Asset asset = assetResource.adaptTo(Asset.class);

String mimeType = asset.getMimeType();
long size = asset.getSize();
Rendition original = asset.getOriginal();
Rendition thumb = asset.getRendition("cq5dam.thumbnail.140.100.png");

// Read metadata
ValueMap metadata = assetResource.getChild("jcr:content/metadata").getValueMap();
String title = metadata.get("dc:title", String.class);
```

■ **In your project:** Your 12 Content Fragments live in DAM as dam:Asset nodes (their cq:model property is what distinguishes them from binary images). You also use a sample image from `/content/dam/we-retail/` for the header logo. Real AEM projects have custom DAM folder structures with author-friendly tags and metadata schemas.

■ **Interview Tip:** If asked *'what happens when you upload an image to DAM?'*: AEM triggers the DAM Update Asset workflow which extracts metadata (EXIF/IPTC), generates thumbnail and web renditions, and indexes the asset. Mentioning the workflow shows you understand the trigger-driven nature of DAM.

20. AEM Workflows

Concept

A **Workflow** is a series of steps automating a content process: review, approval, asset processing, publishing. Each step is either automatic (Process step running Java) or manual (Participant step waiting for human action).

Workflow vocabulary

Term	Meaning
Workflow Model	The blueprint — sequence of steps. Stored under /var/workflow/models/
Workflow Instance	A running execution of a model. Stored under /var/workflow/instances/
Payload	The resource the workflow is operating on (a page, asset)
Step	A single operation in the model
Process Step	Java code (a Process implementation)
Participant Step	Waits for a human to complete a work item
Container Step	Embeds another workflow model
OR / AND Split	Branching control flow

Common OOTB workflows

- **Request for Activation** — request publish approval.
- **Publish Example** — auto-publish on approval.
- **DAM Update Asset** — process uploaded assets.
- **Request for Deletion** — approval before delete.
- **Move/Delete** — bulk operations.

Custom Process step (Java)

```
package com.genc.core.core.workflows;

import com.adobe.granite.workflow.WorkflowSession;
import com.adobe.granite.workflow.exec.WorkItem;
import com.adobe.granite.workflow.exec.WorkflowProcess;
import com.adobe.granite.workflow.metadata.MetaDataMap;
import org.osgi.service.component.annotations.Component;

@Component(
    service = WorkflowProcess.class,
    property = {
        "process.label=GenC - Notify on Publish"
    }
)
public class NotifyOnPublishProcess implements WorkflowProcess {

    @Override
    public void execute(WorkItem workItem, WorkflowSession workflowSession,
                        MetaDataMap metaData) {
        String payloadPath = workItem.getWorkflowData().getPayload().toString();
        // Custom logic: send email, call API, etc.
    }
}
```

How to use the custom step

- Open **Tools** → **Workflow** → **Models**.
- Edit a model, drag a **Process Step**, configure it.
- In the 'Process' dropdown, your `process.label` appears.
- Save → workflow now runs your Java when this step executes.

Triggering a workflow

- **Manually** — Sites Console → page → Workflow → Start Workflow.
- **Launcher** — auto-start when a node changes (Tools → Workflow → Launchers).
- **From code** — `workflowSession.startWorkflow(model, data)`.

■ **Interview Tip:** *'How would you auto-publish a page when an author submits it for review?'*: (1) Create a workflow model with a Participant step (reviewer) → Process step that calls Replicator.replicate(ACTIVATE). (2) Add a launcher on /content/yoursite that fires the workflow when a page's cq:lastModified changes. Mention you'd use a Service User for the replicate call.

21. Replication & Dispatcher

Author → Publisher pipeline

Replication is the process of copying content from the author instance to one or more publish instances. Authors don't directly serve traffic; they publish content, which *replicates* to publishers that serve end users (usually behind a dispatcher cache).

```
[Author 4502]  --replicate-->  [Publisher 4503]  <--cache--  [Dispatcher (Apache)]
                                                                        ^
                                                                        |
                                                                End user request
```

Replication agents

An **agent** is an OSGi-configured connection from author to a publisher. Found at `/etc/replication/agents.author/`. Each agent has:

- **Transport URI** — publisher URL (e.g. http://publish-host:4503/bin/receive).
- **Transport credentials** — username/password to authenticate.
- **Triggers** — when to fire (on modification, on receive, etc.).
- **Serialization type** — what format to send (default Durbo).

Replication actions

Action	What it does
Activate	Publish — push content from author to publisher
Deactivate	Unpublish — remove content from publisher
Delete	Replicate a delete operation
Test	Verify the agent is reachable

Reverse replication

Normally content flows Author → Publish. **Reverse replication** sends data the other way — e.g. user-generated content (comments, form submissions) from publisher back to author. Configured under `/etc/replication/agents.publish/`.

Flush agent

A **Dispatcher Flush agent** tells the dispatcher to invalidate its cache when content changes. After a successful publisher activation, the flush agent fires to the dispatcher's `/dispatcher/invalidate.cache` URL with the path that changed.

Dispatcher

Dispatcher is an Apache HTTP Server module from Adobe that sits in front of publishers. It handles three jobs:

- **Caching** — stores rendered HTML on disk; subsequent requests hit the cache, not AEM. Dramatically reduces load.
- **Load balancing** — round-robins requests across multiple publisher instances.
- **Security** — filters out dangerous URL patterns, hides internal paths like /apps and /system.

Dispatcher config — key files

- **dispatcher.any** — main config with /publishfarm, /cache rules, /filter rules.
- **filters** — whitelist/blacklist URL patterns. Default: deny most, allow .html, .css, .js.
- **cache rules** — which extensions to cache. Default: cache .html and static files, don't cache .json.
- **invalidation** — which paths to invalidate when a flush comes in.

Cache invalidation strategies

- **Path-based** — invalidate the published path and its parents.
- **Stat file** — touch a stat file; anything older than it is considered stale.
- **Time-based** — combine with TTL headers.

■ **Interview Tip:** *'What's between an end user and AEM?'* Dispatcher → load balancer → publisher. Then explain dispatcher does caching + load balancing + security filtering. If asked about *'why caching matters'*: a cached HTML page is served from disk in milliseconds, no JVM cost, lets a small AEM cluster handle very large traffic.

22. Multi-Site Management (MSM)

Concept

MSM lets you build many country/language versions of a site from a single source. You author once in the master, then **rollout** updates to all live copies, with the option to customize per-region.

Key concepts

Concept	Meaning
Blueprint	The source site (a normal AEM site marked as a master).
Live Copy	A copy that maintains a live link to the blueprint and pulls updates.
Language Copy	A copy intended for translation. Same structure, translated content.
Rollout	The action of pushing changes from blueprint to live copies.
Rollout Config	Defines what gets rolled out and how (overwrite, merge, skip).
Inheritance	A page in a live copy inherits from its blueprint; can be cancelled per-component.
Suspension	Pause inheritance temporarily (e.g., during a region-specific campaign).

Example MSM hierarchy

```
/content/we-retail/
■■■ language-masters/      ← Blueprints (authored once)
■   ■■■ en/
■   ■■■ fr/
■   ■■■ de/
■■■ us/, fr/, de/, ...     ← Live Copies (rolled out from masters)
    ■■■ en/
    ■■■ ...
```

Rollout configurations

- **Standard rollout config** — overwrite live copy with blueprint changes.
- **Push on modify** — auto-rollout when blueprint page is edited.
- **Activate on blueprint activation** — auto-publish live copies when blueprint publishes.

MSM vs Language Copy vs Live Copy

- **Live Copy** — keeps a live link, propagates updates from blueprint. Used when content is the same with minor regional tweaks.
- **Language Copy** — created via Translation Workflow, sent to translators. Used when content must be translated.
- **Site/Folder** — a normal copy with no live link. Used for truly independent sites.

■ **Interview Tip:** *'How would you build a multi-country website?'* Answer: blueprint in /content/site/language-masters/en, live copies for each country (us, uk, in, jp), MSM rollout configs to push blueprint updates. Per-country customizations break inheritance on specific components. Translation workflow handles language variants.

23. JUnits & Mockito (Unit Testing)

Why unit test in AEM

Sling Models and OSGi services are plain Java — testable in isolation without running AEM. The **AEM Mocks** framework (from wcm.io) provides in-memory implementations of JCR, Sling, OSGi so you can test code that touches them.

JUnit 5 basics

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import static org.junit.jupiter.api.Assertions.*;

class CourseBeanTest {

    private CourseBean bean;

    @BeforeEach
    void setUp() {
        bean = new CourseBean();
    }

    @Test
    void titleGetterAndSetterWork() {
        bean.setTitle("BCA");
        assertEquals("BCA", bean.getTitle());
    }

    @Test
    void emptyBeanHasNoCategory() {
        assertNull(bean.getCategory());
    }
}
```

Common JUnit 5 annotations

Annotation	When called
@Test	Marks a method as a test
@BeforeEach	Run before each @Test
@AfterEach	Run after each @Test
@BeforeAll	Run once before all tests (static)
@AfterAll	Run once after all tests (static)
@Disabled	Skip this test
@ParameterizedTest	Run with multiple input values
@DisplayName("...")	Human-readable test name

Common assertions

```
assertEquals(expected, actual);
assertNotEquals(unexpected, actual);
assertTrue(condition);
assertFalse(condition);
assertNull(value);
assertNotNull(value);
assertSame(expectedRef, actualRef);     // ==
assertNotSame(unexpectedRef, actualRef);
assertThrows(IOException.class, () -> { method(); });
assertAll(
    () -> assertEquals(1, x),
    () -> assertEquals(2, y)
);
```

Mockito — mocking dependencies

```
import org.mockito.Mock;
import org.mockito.InjectMocks;
import org.mockito.junit.jupiter.MockitoExtension;
import org.junit.jupiter.api.extension.ExtendWith;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class CourseListModelTest {

    @Mock
    private CourseConfigService configService;

    @Mock
    private SlingHttpServletRequest request;

    @InjectMocks
    private CourseListModel model;

    @Test
    void usesOSGiDefaultWhenNoDialogLimit() {
        when(configService.getDefaultLimit()).thenReturn(5);
        // ... arrange more stubs ...
        model.init();
        assertEquals(5, model.getCourses().size());
        verify(configService, times(1)).getDefaultLimit();
    }
}
```

Mockito vocabulary

Construct	Purpose
mock(Cls.class) or @Mock	Create a mock object — all methods return defaults
@InjectMocks	Create real instance, inject mocks into its fields
when(x.foo()).thenReturn(y)	Stub a method call
doThrow(e).when(x).foo()	Stub a method to throw
verify(x).foo()	Assert method was called
verify(x, times(2))	Assert called exactly N times
verify(x, never())	Assert NOT called
ArgumentCaptor	Capture args passed to a method for assertion

AEM Mocks (wcm.io) — for tests that touch JCR

```
import io.wcm.testing.mock.aem.junit5.AemContext;
import io.wcm.testing.mock.aem.junit5.AemContextExtension;
import org.junit.jupiter.api.extension.ExtendWith;

@ExtendWith(AemContextExtension.class)
class CourseListModelTestWithAem {

    private final AemContext ctx = new AemContext();

    @Test
    void readsCourseFragments() {
        ctx.load().json("/sample-content.json", "/content/dam/.../courses");
        ctx.addModelsForClasses(CourseListModel.class);
        ctx.currentResource("/content/dam/.../courses");

        CourseListModel model = ctx.request().adaptTo(CourseListModel.class);
        assertNotNull(model);
        assertEquals(12, model.getTotalCount());
    }
}
```

■ **In your project:** Your project has an **it.tests** module (integration tests, passing) and **ui.tests** (Cypress E2E, skipped due to SSL cert issue). Be honest about the SSL skip if asked — it's a network restriction, not a test design problem. If pressed, say you'd write unit tests for CourseListModel using Mockito to mock CourseConfigService and AemContext to load sample CF fixtures.

■ **Interview Tip:** *'How would you test your CourseListModel?'* Two-level answer: (1) **Unit test** with Mockito — mock CourseConfigService, ResourceResolver, verify the model returns correct list. (2) **Integration test** with AemContext — load sample CF fixtures as JSON, adapt the request to the model, assert behavior. Mention the AAA pattern: Arrange, Act, Assert.

24. RTE (Rich Text Editor) & Advanced Touch UI

RTE — Rich Text Editor

The RTE is the WYSIWYG editor in dialogs and inline editing. It lets authors style text (bold, italic, lists, links). It's a Granite UI component with extensive plugin support.

RTE configuration

```
<text jcr:primaryType="nt:unstructured"
   sling:resourceType="cq/gui/components/authoring/dialog/richtext"
   useFixedInlineToolbar="{Boolean}true"
   fieldLabel="Body"
   name="./text">
   <rtePlugins jcr:primaryType="nt:unstructured">
       <format jcr:primaryType="nt:unstructured" features="bold,italic,underline"/>
       <links  jcr:primaryType="nt:unstructured" features="modifylink,unlink"/>
       <lists  jcr:primaryType="nt:unstructured" features="*"/>
       <paraformat jcr:primaryType="nt:unstructured" features="*">
           <formats jcr:primaryType="cq:WidgetCollection">
               <p  jcr:primaryType="nt:unstructured" tag="p"  description="Paragraph"/>
               <h2 jcr:primaryType="nt:unstructured" tag="h2" description="Heading 2"/>
           </formats>
       </paraformat>
   </rtePlugins>
</text>
```

Common RTE plugins

Plugin	Provides
format	bold, italic, underline
links	insert/edit/remove hyperlinks
lists	bullet and numbered lists
paraformat	heading levels (h1, h2, ...)
justify	left/center/right alignment
image	insert image from DAM
table	create/edit tables
findreplace	find and replace text
spellcheck	spell checking

Advanced Touch UI — Listeners

```
<myField ...>
    <granite:data jcr:primaryType="nt:unstructured"
                  cq-dialog-dropdown-showhide-target=".hidden-section"/>
</myField>

<!-- OR with old-style listeners -->
<listeners jcr:primaryType="nt:unstructured"
           jcr:primaryType="cq:WidgetListener"
           dialogready="function() { /* JS */ }"
           selectionchanged="function(field) { /* JS */ }"/>
```

Conditional show/hide

Use the **cq-dialog-dropdown-showhide** mechanism: a select field has `granite:class='cq-dialog-dropdown-showhide'` and `cq-dialog-dropdown-showhide-target='.css-class'`. Other fields with that class (and a `showhidetargetvalue` data attribute) appear/disappear based on selection.

Validators

```
<emailField jcr:primaryType="nt:unstructured"
    sling:resourceType="granite/ui/components/coral/foundation/form/textfield"
    name="./email"
    fieldLabel="Email"
    required="{Boolean}true"
    validation="my-email-validator"/>
```

Then register a JS validator in a clientlib category `cq.authoring.dialog.all`:

```
(function($, Granite) {
    Granite.UI.Foundation.Registry.register(
        "foundation.validation.validator",
        {
            selector: "[data-validation='my-email-validator']",
            validate: function(el) {
                if (!/^[^@]+@[^@]+\.[^@]+$/.test(el.value)) {
                    return "Please enter a valid email";
                }
            }
        }
    );
})(jQuery, Granite);
```

Datasources (refresher)

A datasource is a Sling Servlet (registered by resource type) that produces a list to populate a select dropdown. Used when options are dynamic (read from JCR, from an external API, or computed).

■ **Interview Tip:** *'How do you show one field only when a checkbox is ticked?'*: use the **cq-dialog-checkbox-showhide** mechanism — checkbox has the marker class, target field has a class that gets toggled. No custom JS needed for the common case.

25. AEM Best Practices & Coding Standards

Component design

- Use **core components** (Adobe's OOTB components) as base classes via sling:resourceSuperType — inherit, don't duplicate.
- Follow the **BEM** naming convention for CSS classes (block__element--modifier).
- Components should be **self-contained** — render correctly with default content.
- Use **_cq_template.xml** for components that use data-sly-use on child resources (avoids first-drop crashes).
- Use the **componentGroup** property — empty group hides from authors (useful for sub-components).

Sling Models

- Always set `defaultInjectionStrategy = OPTIONAL` — robust against missing dialog fields.
- List both Request and Resource as adaptables for flexibility.
- Pre-format dates in Java — HTL has issues with Calendar.
- Put business logic in @PostConstruct, never in getters.
- Use SLF4J logger, never System.out.println.
- Return immutable collections from getters (Collections.unmodifiableList).

OSGi & Java code

- **Never use admin resource resolver** — always go through service users.
- **Always close ResourceResolvers** in try-with-resources blocks.
- Use **@Reference policy = ReferencePolicy.DYNAMIC** for services that may come and go.
- Use **OSGi configurations (.cfg.json)** for environment-varying values, not hard-coded constants.
- Bundle activator/deactivator handlers should be idempotent.

Templates & policies

- Use **editable templates**, not static ones (Adobe deprecated static).
- Lock down structure components at the template level.
- Use policies (not design dialogs) to restrict component choice.
- Reuse policies across templates via the same policy node.

Performance

- Cache aggressively at the dispatcher; design URLs to be cacheable (avoid request-param-driven dynamic content).
- Use **Sling Dynamic Include (SDI)** for user-specific content within otherwise cacheable pages.
- Avoid querying the JCR in HTL — pre-compute in Sling Model.
- Use indexed JCR queries; check `/system/console/jcr-resolver` and `/oak:index`.
- Minify clientlibs; embed small libs rather than load many small files.

Security

- Sanitize all author-provided HTML — use HTL's `@ context='html'`.

- Whitelist URLs at the dispatcher; default-deny.
- Hide /apps, /libs, /system paths from public — dispatcher filter.
- Use service users; never admin session in code.
- Apply principle of least privilege in ACLs.
- Keep AEM patched — Adobe releases service packs and hot fixes.

ACS AEM Commons (worth knowing)

ACS Commons is a community-maintained library of useful AEM utilities. Common features: MCP (multi-process management), redirect manager, mock data, asset generator, link checker. Mentioning it shows you know the ecosystem.

■ **Interview Tip:** If asked *'what's an AEM best practice you follow?'* pick ONE and elaborate. Strongest answer: **configurability over hard-coding** — your CourseConfigService stores the DAM path so admins can change environments without redeploying. Tie it to your case study.

26. Issues You Faced — Be Ready to Talk About These

Your case study documented several real issues you hit and fixed. Interviewers love these stories because they show problem-solving. Practice articulating each in two sentences: (1) what went wrong, (2) what you did.

Issue	Cause	Fix you applied
Malformed filter.xml	Manual edit typo wrote <__workspaceFilter>	Replaced with correct <workspaceFilter>
/conf not allowed in ui.apps	Wrong module ownership	Moved /conf and /cq:tags filters to ui.content
Maven could not delete generated HTL Java file	Eclipse auto-build held file lock	Disabled Eclipse auto-build, manually cleaned target/
HTL ">" not allowed inside ${} expression	HTL parser limitation	Removed inline comparison; logic moved to Sling Model
Dates rendering as Calendar.toString()	HTL @format on Calendar unreliable	Pre-formatted as String "dd MMM yyyy" in CourseBean
Course categoryTitle empty	TagManager returned null for namespace mismatch	Added fallback: extract last segment, capitalize
Multifield items not rendering	Composite multifield stores child nodes, not JSON	Used data-sly-use + .children pattern
Footer crash on fresh drop	No use provider for empty footerLinks child	Added _cq_template.xml to pre-create the child node
HTL mismatched input (	Method calls with args not allowed in HTL 1.4	Used data-sly-use pattern instead
Cypress UI tests fail every build	SSL cert verification can't reach nodejs.org from container	Documented as network restriction; not relevant to functi

Story formula

'I was building [X]. When I tried [Y], I saw [error]. I diagnosed it by [Z]. The root cause was [understanding]. I fixed it by [solution].' Practice this for the top 3 issues — pick the ones with the most learning.

■ **Interview Tip:** Strongest story is the **composite multifield + _cq_template.xml** bug. It shows you understand: (1) how multifield stores data differently in composite mode, (2) how HTL's data-sly-use needs a real resource, (3) how AEM components have a first-drop initial state. Three layers of AEM knowledge in one fix.

PART 3 — Java Fundamentals

Cognizant evaluations always include Java questions alongside AEM. This part covers OOP, Collections, Exceptions, Java 8 streams/lambdas/functional interfaces, and threading basics. All examples are short and interview-ready.

27. OOP — Four Pillars

1. Encapsulation

Bundle data (fields) and the methods that operate on them into a class, hide internal state with `private`, expose access through public getters/setters. Lets you change implementation without breaking callers.

```
public class Course {
    private String title;
    private int duration;

    public String getTitle() { return title; }
    public void setTitle(String title) {
        if (title == null || title.isBlank())
            throw new IllegalArgumentException("Title required");
        this.title = title;
    }
}
```

2. Inheritance

A class can extend another, gaining its fields and methods. Express an *is-a* relationship. Single class inheritance; multiple interface inheritance.

```
class Vehicle {
    protected int speed;
    void start() { ... }
}

class Car extends Vehicle {
    int wheels = 4;
    @Override
    void start() {
        super.start();
        System.out.println("Car engine on");
    }
}
```

3. Polymorphism

- **Compile-time (overloading)** — same method name, different parameter list.
- **Run-time (overriding)** — child class redefines a method from parent; JVM picks impl based on actual object type.

```
// Overloading
class Calc {
    int add(int a, int b)         { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c)  { return a + b + c; }
}

// Overriding
Vehicle v = new Car();
v.start();   // Calls Car.start() — run-time decision
```

4. Abstraction

Hide complex implementation, expose simple interface. Achieved via **abstract classes** (can have state and concrete methods) and **interfaces** (contracts only, with default methods since Java 8).

```
abstract class Shape {
    abstract double area();              // Must be implemented by subclass
    void describe() {                    // Concrete method
        System.out.println("Area: " + area());
    }
}
```

```
interface Drawable {
    void draw();                          // Implicitly public abstract
    default void render() {               // Java 8: default method
        System.out.println("Rendering...");
        draw();
    }
}

class Circle extends Shape implements Drawable {
    double r;
    @Override double area() { return Math.PI * r * r; }
    @Override public void draw() { ... }
}
```

Abstract class vs Interface

Aspect	Abstract Class	Interface
Methods	Can have abstract + concrete	All abstract by default; default + static since Java 8; private sir
Fields	Instance fields allowed	Only public static final (constants)
Constructor	Yes	No
Multiple inheritance	No (one class)	Yes (many interfaces)
When to use	Share code among related classes	Define a contract many unrelated classes implement

this vs super

- **this** — refers to the current object. Used to disambiguate fields from parameters, or call another constructor in the same class via `this(...)`.
- **super** — refers to the parent. Used to call parent method (`super.method()`) or parent constructor (`super(...)`, must be the first line).

static keyword

- **static field** — one copy shared across all instances. Class-level state.
- **static method** — called on the class, not an instance. Cannot access instance fields directly.
- **static block** — runs once when the class is loaded.
- **static nested class** — doesn't need an enclosing instance to instantiate.

final keyword

- **final variable** — assigned once. If primitive, value can't change; if object reference, reference can't change (object's contents still can).
- **final method** — can't be overridden by subclass.
- **final class** — can't be subclassed (e.g. String).

■ **Interview Tip:** Classic question: *'Difference between method overloading and overriding?'* Overloading is compile-time, same name + different params, within one class. Overriding is run-time, same signature, child overrides parent. Followup: *'can you override a static method?'* No, you can **hide** it (define a static method with the same signature in the child), but it's resolved by the declared type, not runtime type.

28. Java Collections Framework

The hierarchy

```
Iterable
■■■ Collection
    ■■■ List         (ordered, allows duplicates)
    ■   ■■■ ArrayList     (resizable array — fast random access)
    ■   ■■■ LinkedList    (doubly-linked list — fast insert/delete)
    ■   ■■■ Vector        (legacy synchronized list)
    ■■■ Set          (no duplicates)
    ■   ■■■ HashSet       (hash-based — O(1) avg)
    ■   ■■■ LinkedHashSet (insertion order)
    ■   ■■■ TreeSet       (sorted, O(log n))
    ■■■ Queue        (FIFO operations)
        ■■■ PriorityQueue (heap, ordered by priority)
        ■■■ ArrayDeque    (double-ended queue)

Map (separate hierarchy, not extending Collection)
■■■ HashMap         (hash-based)
■■■ LinkedHashMap   (insertion order)
■■■ TreeMap         (sorted by key)
■■■ Hashtable       (legacy synchronized)
```

List comparison

Op	ArrayList	LinkedList
get(i)	O(1)	O(n)
add(end)	O(1) amortized	O(1)
add(mid)	O(n)	O(n) for find + O(1) for insert
remove(i)	O(n)	O(n) find + O(1) remove
Memory	Compact (array)	Higher (each node has 2 refs)
Use when	Lots of reads, append-mostly	Lots of inserts/removes in middle

HashMap internals (very common interview topic)

- Backed by an array of **buckets**; each bucket holds a linked list (or, since Java 8, a balanced tree when length exceeds 8).
- **put(k, v)**: compute `k.hashCode()` → modulo bucket count → walk that bucket's chain, comparing via `equals()`.
- **Load factor** default 0.75 — when filled fraction exceeds this, the array doubles and all entries are **rehashed**.
- **Initial capacity** default 16. Set higher in constructor if you know the size to avoid rehashing.
- Allows one null key and many null values. **Hashtable** allows neither (and is synchronized).

equals() & hashCode() contract

- If two objects are equal (`a.equals(b)`), they MUST have the same hashCode.
- If hashCodes differ, the objects must not be equal.
- Two unequal objects MAY share a hashCode (collision is allowed).
- Violating this breaks HashSet/HashMap lookups silently.

```
public class Course {
```

```
    private String code;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Course)) return false;
        return Objects.equals(code, ((Course) o).code);
    }

    @Override
    public int hashCode() {
        return Objects.hash(code);
    }
}
```

Comparable vs Comparator

```
// Comparable — natural ordering, implemented by the class
class Course implements Comparable<Course> {
    private int duration;
    @Override
    public int compareTo(Course other) {
        return Integer.compare(this.duration, other.duration);
    }
}

// Comparator — external ordering, separate class
Comparator<Course> byTitle = Comparator.comparing(Course::getTitle);
courses.sort(byTitle.reversed());
```

Iterator vs ListIterator vs Enhanced for

- **Enhanced for (for-each)** — read-only traversal, can't modify the collection.
- **Iterator** — supports remove() during iteration; one-directional.
- **ListIterator** — supports add/set/remove and bidirectional traversal; List-only.

Fail-fast vs Fail-safe iterators

- **Fail-fast** — throw `ConcurrentModificationException` if the collection is modified during iteration. ArrayList, HashMap, HashSet.
- **Fail-safe** — iterate over a snapshot, no exception. CopyOnWriteArrayList, ConcurrentHashMap.

Concurrent collections

Class	Use case
ConcurrentHashMap	Thread-safe Map; segment-locked for high throughput.
CopyOnWriteArrayList	Reads vastly outnumber writes; iteration never blocks.
BlockingQueue (ArrayBlockingQueue, LinkedBlockingQueue)	Producer-consumer patterns.

■ **Interview Tip:** Top collections question: *'How does HashMap work internally?'* Walk through: array of buckets → hashCode → bucket index → walk chain comparing with equals → load factor 0.75 triggers rehash → since Java 8 a bucket converts to a red-black tree at size 8. Mention that bad hashCode = O(n) lookups in the worst case.

29. Exceptions & Error Handling

Exception hierarchy

```
Throwable
■■■ Error                    (JVM-level, don't catch — OutOfMemoryError, StackOverflowError)
■■■ Exception
    ■■■ RuntimeException     (UNCHECKED — compiler doesn't force handling)
    ■   ■■■ NullPointerException
    ■   ■■■ ArrayIndexOutOfBoundsException
    ■   ■■■ ClassCastException
    ■   ■■■ IllegalArgumentException
    ■   ■■■ IllegalStateException
    ■   ■■■ NumberFormatException
    ■   ■■■ ArithmeticException
    ■■■ (Checked exceptions — compiler enforces handling)
        ■■■ IOException
        ■■■ SQLException
        ■■■ ClassNotFoundException
        ■■■ InterruptedException
```

Checked vs Unchecked — when to use each

- **Checked** (extends Exception, NOT RuntimeException) — recoverable conditions caller should handle. e.g., file not found, network down. Compiler forces try/catch or throws.
- **Unchecked** (extends RuntimeException) — programming errors that shouldn't be caught everywhere. e.g., null deref, illegal arg. Compiler does not require handling.

try / catch / finally

```
try {
    Resource r = openResource();
    process(r);
} catch (IOException e) {
    LOG.error("Read failed", e);
    throw new RuntimeException(e);     // Wrap & rethrow
} catch (IllegalStateException | NumberFormatException e) {
    LOG.warn("Bad input", e);          // Multi-catch (Java 7+)
} finally {
    LOG.info("Done");                  // Always runs (unless JVM exits)
}
```

try-with-resources (Java 7+)

Auto-closes anything implementing `AutoCloseable`. Replaces verbose finally blocks. Multiple resources comma-separated; closed in reverse order.

```
try (BufferedReader r = new BufferedReader(new FileReader("a.txt"));
     PrintWriter   w = new PrintWriter("b.txt")) {
    String line;
    while ((line = r.readLine()) != null) w.println(line);
}   // r and w both closed here automatically
```

throw vs throws

- **throw** — verb. Used inside a method to actually raise an exception. `throw new IOException("bad")`.
- **throws** — declaration. Used in a method signature to declare which checked exceptions may escape. `void read() throws IOException { ... }`.

Custom exceptions

```
public class CourseNotFoundException extends RuntimeException {
    public CourseNotFoundException(String code) {
        super("Course not found: " + code);
    }
    public CourseNotFoundException(String code, Throwable cause) {
        super("Course not found: " + code, cause);
    }
}
```

Common mistakes

- **Catch and ignore** — never `catch(Exception e) {}`. At minimum, log it.
- **Catching Exception** too broadly — catch specific types so you can handle them differently.
- **Throwing Exception** as the declared type — too vague; declare specific types.
- **Returning null from finally** — finally can override a return from try silently. Avoid returns in finally.
- **Closing resources in finally without null check** — use try-with-resources instead.

■ **Interview Tip:** Likely question: *'Will the finally block execute if the try block returns?'* Yes. Even if try returns or throws, finally still runs. The only ways finally is skipped: JVM exits (System.exit), JVM crashes, or the thread is killed.

30. Java 8 — Lambdas, Streams, Functional Interfaces

Functional Interface

An interface with exactly ONE abstract method. The @FunctionalInterface annotation is optional but recommended (compiler enforces the constraint). Examples in java.util.function:

Interface	Signature	Use
Function<T,R>	R apply(T t)	Convert T to R
Predicate<T>	boolean test(T t)	Test a condition
Consumer<T>	void accept(T t)	Consume a value
Supplier<T>	T get()	Produce a value
BiFunction<T,U,R>	R apply(T, U)	Two inputs, one output
UnaryOperator<T>	T apply(T)	Function<T,T>
BinaryOperator<T>	T apply(T,T)	Reducer

Lambda syntax

```
// (params) -> expression
Runnable r = () -> System.out.println("Hello");
Function<Integer, Integer> sq = x -> x * x;
BiFunction<Integer, Integer, Integer> sum = (a, b) -> a + b;

// (params) -> { statements; return value; }
Comparator<String> byLen = (a, b) -> {
    int diff = a.length() - b.length();
    return diff != 0 ? diff : a.compareTo(b);
};
```

Method references — shorthand for lambdas

```
// Lambda                              // Equivalent method reference
list.forEach(s -> System.out.println(s));  list.forEach(System.out::println);
list.stream().map(s -> s.toUpperCase());    list.stream().map(String::toUpperCase);
list.sort((a, b) -> Integer.compare(a, b)); list.sort(Integer::compare);
() -> new ArrayList<>();                   ArrayList::new;
```

Streams — pipeline of operations

```
List<Course> courses = ...;

// Filter, transform, collect — declarative
List<String> engineeringTitles = courses.stream()
    .filter(c -> c.getCategory().equals("Engineering"))
    .map(Course::getTitle)
    .sorted()
    .collect(Collectors.toList());

// Sum a numeric field
int totalCredits = courses.stream()
    .mapToInt(Course::getCredits)
    .sum();

// Group by category
Map<String, List<Course>> byCategory = courses.stream()
    .collect(Collectors.groupingBy(Course::getCategory));

// Count by category
Map<String, Long> counts = courses.stream()
    .collect(Collectors.groupingBy(Course::getCategory, Collectors.counting()));
```

Intermediate vs Terminal operations

Intermediate (lazy, return Stream)	Terminal (eager, end the pipeline)
filter, map, flatMap	collect, forEach, count, sum
sorted, distinct, limit, skip	reduce, min, max, anyMatch
peek (debugging)	allMatch, noneMatch, findFirst, findAny, toArray

Optional — replacing nulls

```
Optional<Course> maybe = repository.findByCode("BCA");

// Bad — same as null check:
if (maybe.isPresent()) { Course c = maybe.get(); ... }

// Good — functional style:
maybe.map(Course::getTitle)
     .ifPresent(System.out::println);

String title = maybe.map(Course::getTitle).orElse("Unknown");
String title2 = maybe.map(Course::getTitle).orElseThrow(
    () -> new CourseNotFoundException("BCA"));
```

default & static methods in interfaces

```
interface Renderer {
    void render();                                  // abstract

    default void renderTwice() {                    // default - has body
        render();
        render();
    }

    static Renderer noop() {                        // static - on the interface
        return () -> {};
    }
}
```

LocalDate / LocalTime (java.time)

```
LocalDate today = LocalDate.now();
LocalDate dob   = LocalDate.of(1995, 6, 15);
LocalDate plus30 = today.plusDays(30);

Period age = Period.between(dob, today);
int years = age.getYears();

LocalDateTime now = LocalDateTime.now();
String formatted = now.format(DateTimeFormatter.ofPattern("dd MMM yyyy"));
```

■ **Interview Tip:** *'Why streams over for loops?'* Three reasons: (1) declarative — say WHAT not HOW, (2) chainable / composable — multi-step transformations read top-to-bottom, (3) parallel — `.parallelStream()` uses ForkJoinPool for free. Caveat: streams are SLOWER for tight numeric loops; use arrays/IntStream for hot paths.

31. Threads & Concurrency (quick refresher)

Creating threads — three ways

```
// 1. Extend Thread
class Worker extends Thread {
    public void run() { System.out.println("Working"); }
}
new Worker().start();

// 2. Implement Runnable (preferred – class is free to extend something else)
Runnable task = () -> System.out.println("Working");
new Thread(task).start();

// 3. ExecutorService (preferred for real work – manages a pool)
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(task);
pool.shutdown();
```

Thread lifecycle states

- **NEW** — created but not yet started.
- **RUNNABLE** — eligible to run; JVM scheduler decides when.
- **BLOCKED** — waiting to acquire a monitor lock.
- **WAITING** — paused indefinitely (wait(), join() without timeout).
- **TIMED_WAITING** — paused for a specific duration (sleep(ms), wait(ms)).
- **TERMINATED** — run() has completed.

synchronized

Mutual exclusion — only one thread can hold the lock for an object at a time.

```
// Method-level: locks 'this'
public synchronized void increment() { count++; }

// Static method: locks the Class object
public static synchronized int getCount() { return count; }

// Block-level: lock any object
public void deposit(double amount) {
    synchronized (this) { balance += amount; }
}
```

volatile

Tells the JVM that a field is shared between threads — reads always go to main memory, writes are immediately published. Lighter than synchronized but only ensures visibility, not atomicity. Use for boolean flags.

```
private volatile boolean running = true;
// Thread A
while (running) { doWork(); }
// Thread B
running = false;     // Guaranteed visible to A immediately
```

AtomicInteger and friends

```
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet();        // Thread-safe ++count
count.compareAndSet(5, 10);     // CAS
```

wait / notify / notifyAll

Used WITHIN a synchronized block on the same monitor. wait() releases the lock and pauses; notify wakes one waiting thread, notifyAll wakes all.

■ **Interview Tip:** *'Difference between sleep() and wait()?'* sleep is Thread.sleep — pauses the thread but doesn't release any locks; wait is Object.wait — releases the lock and waits for notify. sleep is for timing, wait is for thread coordination.

PART 4 — Web Fundamentals

HTML, CSS, JavaScript, plus Servlets & JSP. These appear in Cognizant evaluations alongside AEM — be ready for short coding questions like *'write a servlet that handles a form'* or *'how do you center a div'*.

32. HTML5 Essentials

Document structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page Title</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>...</header>
    <nav>...</nav>
    <main>
        <section>
            <article>...</article>
        </section>
        <aside>...</aside>
    </main>
    <footer>...</footer>
    <script src="app.js"></script>
</body>
</html>
```

Semantic elements (HTML5)

Element	Use
<header>	Top of a page or section — logo, intro
<nav>	Primary navigation links
<main>	Dominant content — exactly one per page
<section>	Thematic grouping with a heading
<article>	Self-contained content (blog post, card)
<aside>	Sidebar, related links
<footer>	Bottom of a page or section
<figure>/<figcaption>	Image with caption
<time>	Machine-readable date/time

Forms

```
<form action="/submit" method="POST">
    <label for="name">Name</label>
    <input type="text" id="name" name="name" required minlength="2">

    <label for="email">Email</label>
    <input type="email" id="email" name="email" required>

    <label for="course">Course</label>
    <select id="course" name="course">
        <option value="bca">BCA</option>
        <option value="mba">MBA</option>
    </select>

    <fieldset>
        <legend>Mode</legend>
        <input type="radio" id="online" name="mode" value="online"> <label for="online">Online</label>
        <input type="radio" id="offline" name="mode" value="offline"> <label for="offline">Offline</label>
    </fieldset>

    <input type="checkbox" id="agree" name="agree" required>
    <label for="agree">I agree</label>

    <button type="submit">Submit</button>
</form>
```

HTML5 input types

`text`, `email`, `url`, `tel`, `number`, `range`, `date`, `time`, `datetime-local`, `month`, `week`, `color`, `file`, `search`, `password`, `hidden`. Browser provides validation and the appropriate keyboard on mobile.

Accessibility (a11y) basics

- Always use `<label for='id'>` with form controls.
- Provide `alt` text for images.
- Use semantic elements over generic `<div>`.
- Add `aria-*` attributes when semantics aren't enough.
- Make sure focus order is logical; use `tabindex` sparingly.

■ **Interview Tip:** Likely question: *'What's the difference between <section> and ?'* section has semantic meaning — assistive tech announces it as a section, search engines understand structure. div is purely a styling hook. Use section when content has a heading and forms a thematic group.

33. CSS3 — Selectors, Box Model, Flexbox, Grid

Selectors

Selector	Matches
*	Universal — every element
div	Type — all divs
.card	Class
#main	ID (unique)
div.card	div elements with class card
div, p	div OR p (grouping)
div p	p descendants of div (any depth)
div > p	p direct children of div
div + p	p immediately after div (adjacent sibling)
div ~ p	p siblings after div
[type="text"]	Attribute equals
[href^="https"]	href starting with https
:hover, :focus, :active	Pseudo-classes (state)
:first-child, :last-child, :nth-child(2n)	Structural pseudo-classes
::before, ::after	Pseudo-elements

Specificity (which rule wins)

Specificity = (inline style, IDs, classes/attrs/pseudo-classes, types). Higher tuple wins; ties go to the later rule. `!important` overrides everything (use sparingly).

Box model

```
/* By default, width = content only. */
.box {
    width: 200px;       /* content area */
    padding: 10px;      /* inside the border */
    border: 2px solid;  /* the border */
    margin: 20px;       /* outside the border */
}
/* Total occupied: 200 + 2*10 + 2*2 + 2*20 = 264px wide */

/* Better — width includes padding + border */
* { box-sizing: border-box; }
```

Display values

- **block** — full-width, line-broken (div, p, h1).
- **inline** — flows in text, no width/height (span, a).
- **inline-block** — flows like inline, accepts width/height.
- **flex** — flex container (children become flex items).

- **grid** — grid container.
- **none** — removed from layout entirely.

Flexbox — one-dimensional layout

```
.container {
    display: flex;
    flex-direction: row;            /* row | column | row-reverse | column-reverse */
    justify-content: space-between; /* main-axis alignment */
    align-items: center;            /* cross-axis alignment */
    gap: 16px;
    flex-wrap: wrap;
}

.item {
    flex: 1;            /* grow:1 shrink:1 basis:0 — equal-width children */
    /* OR */
    flex: 0 0 200px;    /* fixed 200px width */
}
```

Grid — two-dimensional layout

```
.container {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
    grid-template-rows: auto;
    gap: 20px;
}
/* Responsive: as many 300px+ columns as fit, each at least 1fr */
```

Position values

- **static** — default, in normal flow.
- **relative** — in normal flow + offset; creates a new positioning context.
- **absolute** — out of flow, positioned relative to nearest positioned ancestor.
- **fixed** — out of flow, positioned relative to viewport (stays on scroll).
- **sticky** — relative until it hits a scroll threshold, then fixed.

Responsive design

```
/* Mobile-first */
.card { width: 100%; }

@media (min-width: 768px) {     /* tablet+ */
    .card { width: 48%; }
}
@media (min-width: 1024px) {    /* desktop+ */
    .card { width: 32%; }
}
```

Common modern features

- **CSS variables**: `--brand: #003366; color: var(--brand);`
- **Transitions**: `transition: all 0.3s ease;` + change a property on hover.
- **Transforms**: `transform: translateY(-4px) rotate(5deg);`
- **Gradients**: `background: linear-gradient(135deg, #003366, #0066cc);`
- **Calc()**: `width: calc(100% - 40px);`

■ **Interview Tip:** Likely question: *'How do you center a div?'* Three ways: (1) flex on parent: `display:flex; justify-content:center; align-items:center;`, (2) grid: `display:grid;`

`place-items:center;`, (3) margin auto for horizontal only: `margin: 0 auto;` (requires width).

34. JavaScript Essentials

var vs let vs const

Keyword	Scope	Reassignable	Hoisted
var	Function	Yes	Yes (to undefined)
let	Block	Yes	Yes but TDZ (temporal dead zone)
const	Block	No (but object contents can mutate)	Yes but TDZ

Functions — three syntaxes

```
// 1. Function declaration — hoisted
function greet(name) { return "Hi " + name; }

// 2. Function expression — assigned to a variable
const greet2 = function(name) { return "Hi " + name; };

// 3. Arrow function — concise, no own 'this'
const greet3 = name => "Hi " + name;
const sum    = (a, b) => a + b;
const noop   = () => {};
```

== vs ===

- **==** — loose equality, type coercion. `1 == "1"` is true. Confusing.
- **===** — strict equality, no coercion. `1 === "1"` is false. **Always prefer ===**.

Truthy / Falsy

Falsy values: `false, 0, -0, 0n, '', null, undefined, NaN`. Everything else (including `[]` and `{}`!) is truthy.

this keyword (the JS gotcha)

- Inside a regular function called as a method: `this` = the object it was called on.
- Inside a regular function called standalone: `this` = undefined (strict mode) or global.
- Inside an arrow function: `this` = the surrounding scope's this — arrows don't have their own.
- With `.call()`/`.apply()`/`.bind()`: explicitly set.

Arrays — common operations

```
const arr = [1, 2, 3, 4, 5];

// Transformation
arr.map(x => x * 2);              // [2, 4, 6, 8, 10]
arr.filter(x => x % 2 === 0);     // [2, 4]
arr.reduce((sum, x) => sum + x, 0); // 15

// Iteration
arr.forEach(x => console.log(x));

// Searching
arr.find(x => x > 3);             // 4
arr.findIndex(x => x > 3);        // 3
arr.includes(2);                  // true

// Mutation
arr.push(6); arr.pop();
arr.unshift(0); arr.shift();
arr.splice(1, 2);                 // remove 2 starting at index 1
arr.sort((a, b) => a - b);

// Non-mutating versions
const sorted = [...arr].sort();   // copy first
const sliced = arr.slice(1, 3);
```

Promises & async/await

```
// Promise — async result
fetch('/api/courses')
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.error(err));

// async/await — same thing, looks synchronous
async function loadCourses() {
    try {
        const res  = await fetch('/api/courses');
        const data = await res.json();
        console.log(data);
    } catch (err) {
        console.error(err);
    }
}
```

DOM manipulation

```
// Select
const btn = document.getElementById('save');
const items = document.querySelectorAll('.item');
const first = document.querySelector('.item');

// Modify
btn.textContent = 'Click me';
btn.classList.add('active');
btn.classList.toggle('disabled');
btn.style.backgroundColor = '#003366';

// Create + insert
const div = document.createElement('div');
div.textContent = 'Hello';
document.body.appendChild(div);

// Listen
btn.addEventListener('click', e => {
    e.preventDefault();
    console.log('clicked');
});
```

Event loop (quick concept)

JS is single-threaded but non-blocking. Async work (fetch, setTimeout, promises) is scheduled on the **task queue** (or microtask queue for promises). The **event loop** picks the next task when the call stack is empty. Microtasks run before regular tasks.

ES6+ features

- **Template literals**: `` `Hello ${name}` ``
- **Destructuring**: `const { a, b } = obj; const [x, y] = arr;`
- **Spread / rest**: `const merged = {...a, ...b}; function fn(...args) {}`
- **Default params**: `function f(x = 10) {}`
- **Modules**: `import x from './mod.js'; export default x;`
- **Optional chaining**: `user?.address?.city`
- **Nullish coalescing**: `x ?? 'default'` (vs `||` which also catches 0/empty string)

■ **Interview Tip:** Top JS question: *'What's the difference between == and ===?'* == coerces types (1 == '1' is true), === doesn't (1 === '1' is false). Always use === unless you specifically want coercion. Follow-up: *'Why does typeof null return "object"?'* A 50-year-old bug from JS's origins — null is technically a primitive, not an object, but typeof reports 'object' for historical reasons.

35. Java Servlets & JSP (Web Fundamentals)

Even though AEM uses HTL instead of JSP, evaluators still ask servlet/JSP fundamentals — they're foundational Java web concepts. Sling Servlets (covered in §16) extend the same Servlet API.

What is a Servlet

A **Servlet** is a Java class that handles HTTP requests on a web server. It extends `HttpServlet` and overrides `doGet()`, `doPost()`, etc. Runs inside a servlet container (Tomcat, Jetty, GlassFish — or AEM's embedded container).

Servlet lifecycle

Method	When called	Purpose
init(ServletConfig)	Once, when servlet first loaded	Initialize resources (DB connection, configs)
service(request, response)	Once per request	Dispatches to doGet/doPost based on HTTP method
doGet / doPost / etc.	Once per matching request	Handle the actual request
destroy()	Once, on shutdown / undeploy	Cleanup

Minimal servlet

```
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;
import java.io.IOException;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        res.setContentType("text/html");
        res.getWriter().write("<h1>Hello " + req.getParameter("name") + "</h1>");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        String name = req.getParameter("name");
        // ...process form...
        res.sendRedirect("/thanks.jsp");
    }
}
```

Registering a servlet — two ways

- **@WebServlet annotation** (Servlet 3.0+) — modern, no XML needed.
- **web.xml** deployment descriptor — legacy, more verbose but allows centralized config.

```
<!-- web.xml example -->
<servlet>
    <servlet-name>hello</servlet-name>
    <servlet-class>com.example.HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>hello</servlet-name>
    <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

ServletContext vs ServletConfig

Aspect	ServletConfig	ServletContext
Scope	One servlet	Entire web application
Init params	<init-param> inside <servlet>	<context-param> at <web-app> level
Use	Per-servlet settings	App-wide shared resources
Access	getServletConfig()	getServletContext()

HttpSession — tracking users across requests

```
// In doPost — login
HttpSession session = req.getSession();    // creates if needed
session.setAttribute("user", userObj);
session.setMaxInactiveInterval(30 * 60);   // 30 min timeout

// In doGet — check login
HttpSession session = req.getSession(false);   // don't create
if (session == null || session.getAttribute("user") == null) {
    res.sendRedirect("/login.jsp");
    return;
}

// Logout
session.invalidate();
```

Request scopes — request, session, application

Scope	Lifetime	Use
request	Single HTTP request	Data passed to a JSP via forward
session	One user across multiple requests	Logged-in user, shopping cart
application (context)	Whole app, all users	Config, cache, shared resources

Forward, Include, Redirect

- **RequestDispatcher.forward(req, res)** — server-side hand-off. Browser URL doesn't change. Same request continues.
- **RequestDispatcher.include(req, res)** — embed another resource's output. Used in JSP composition.
- **response.sendRedirect(url)** — client-side: server sends HTTP 302, browser makes a new request. URL changes.

```
// Forward — same request
RequestDispatcher rd = req.getRequestDispatcher("/result.jsp");
rd.forward(req, res);

// Redirect — new request, URL changes
res.sendRedirect("/login.jsp");
```

JSP — JavaServer Pages

JSP is HTML with embedded Java. At first request, the servlet container **compiles** the JSP into a Servlet (its generated Java extends HttpJspBase). Subsequent requests run the compiled servlet.

JSP syntax

```
<%-- JSP comment, not sent to browser --%>

<%@ page import="java.util.*" %>          <%-- Page directive --%>
<%@ taglib uri="..." prefix="c" %>        <%-- Taglib directive --%>

<%! int counter = 0; %>                   <%-- Declaration (instance field) --%>

<% counter++; %>                           <%-- Scriptlet (Java code) --%>

<%= counter %>                             <%-- Expression (writes to output) --%>

${user.name}                               <%-- EL — Expression Language --%>

<c:if test="${count > 0}">                 <%-- JSTL — JSP Standard Tag Library --%>
    <p>Found ${count} items</p>
</c:if>

<c:forEach items="${courses}" var="c">
    <p>${c.title}</p>
</c:forEach>
```

Implicit JSP objects

Variable	Type	Use
request	HttpServletRequest	The HTTP request
response	HttpServletResponse	The HTTP response
session	HttpSession	User session
application	ServletContext	App-wide context
out	JspWriter	Write to response
pageContext	PageContext	Access all scopes
config	ServletConfig	Per-servlet config
page	Object (this)	The JSP instance
exception	Throwable	Only on error pages

MVC pattern with Servlet + JSP

- **Model** — POJO holding data (e.g., Course bean).
- **View** — JSP that renders data using EL/JSTL.
- **Controller** — Servlet that handles the request, populates the model, forwards to the JSP.

```
// Controller (servlet)
@WebServlet("/courses")
public class CourseListServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws IOException, ServletException {
        List<Course> courses = courseService.findAll();
        req.setAttribute("courses", courses);           // Add to request scope
        req.getRequestDispatcher("/WEB-INF/courses.jsp").forward(req, res);
    }
}

// View (courses.jsp) — read from request scope via EL
<c:forEach items="${courses}" var="c">
    <article>
```

```
        <h3>${c.title}</h3>
        <p>${c.description}</p>
    </article>
</c:forEach>
```

JSP vs HTL — why AEM moved away from JSP

Aspect	JSP	HTL
Logic in template	Allowed (scriptlets) — easy to abuse	Forbidden — separation enforced
XSS protection	Manual — easy to forget c:out	Automatic, context-aware
Designer-readable	Harder (scriptlets, taglibs)	Easy (just HTML + data-sly-*)
Performance	Compiled to servlet	Pre-compiled to bytecode
Industry trend	Legacy	Modern (AEM, Adobe-recommended)

■ **Interview Tip:** *'Walk me through what happens when a user submits a form to a servlet.'* (1) Browser sends POST with form fields. (2) Container finds matching servlet by URL pattern. (3) Calls service() which dispatches to doPost(). (4) doPost reads parameters via req.getParameter, validates, persists. (5) Either forward to a JSP (same request) or redirect (new request, PRG pattern — Post/Redirect/Get to avoid duplicate submit on refresh).

36. Mock Interview Q&A; — Drill These

Mix of questions from your case study's anticipated Q&A; plus common evaluator questions. Cover them out loud — saying answers is different from reading them.

Project & Case Study Questions

#	Question	Strong Answer (key points)
1	Walk me through your project in 2 minutes.	5 layers: Tags (genc-aem_training namespace + 10 categories) → CF Model (Cou
2	Why did you create an OSGi config (CourseConfigService) instead of hardcoding the DAM path?	Configurability. The DAM root path can change between environments
3	How does the Course Cards component decide what to show?	Three-way precedence: (1) component dialog values (limit, filterCategory) — most
4	Why are Header and Footer in Experience Fragments instead of hardcoded in the page template?	Single source of truth + author control. Changing the navigation in one XF updates
5	How does your RBAC setup enforce that authors can't edit headers/footers?	Three groups. genc-authors has full rights on /content and /content/dam bu
6	You said your tagfield is rooted at the namespace — what does that achieve?	Scoping. Without a root path, the picker would show every tag in the system (includi
7	Tell me about a bug you fixed in this project.	Pick the composite multifield + _cq_template.xml story. (1) After dropping the Foot
8	Why didn't you publish to a publisher?	Scope of the case study was authoring + custom component dev. In production wo
9	Why are Cypress tests skipped?	Network restriction — the Cypress install step requires downloading binaries from
10	How would you scale this to support 1000 courses?	Pagination + indexed query. Replace the full DAM walk with a JCR query on cq:m

AEM Core Questions

#	Question	Answer hooks
11	What is AEM?	Adobe's enterprise WCM, built on Java + Sling + OSGi + JCR. Author/Publisher/D
12	What is Sling Resource Resolution?	URL → JCR path → sling:resourceType → script at /apps/{type}/{name}.html → H
13	OSGi config vs Sling Model dialog field — when to use which?	OSGi for environment-level admin settings; @ValueMapValue for per-component-
14	Explain Sling Model adaptables.	The Java types you can adapt FROM. Request gives request context; Resource is
15	CF vs XF?	CF = headless data (any channel). XF = authored UI block (page reuse). One is c
16	What does sling:resourceSuperType do?	Inheritance pointer. Resolves to the parent component for inherited scripts/dialog.
17	How does HTL prevent XSS?	Context-aware auto-escaping. Output in attribute context is quote-escaped, in URI
18	What's in /apps vs /content vs /conf?	/apps = immutable code (components, clientlibs). /content = mutable authored dat
19	What does crx:replicate grant?	Permission to publish (replicate) to publisher. NOT included in jcr:all — separate A
20	Editable vs Static template?	Editable: authored in UI, policy-driven, structure changes propagate. Static: develo

Java Questions

#	Question	Answer hooks
21	ArrayList vs LinkedList — which when?	ArrayList: random-access reads, append-mostly. LinkedList: lots of inserts/remove
22	HashMap internals.	Array of buckets, hashCode mod size, linked list per bucket, tree if bucket > 8 (Jav
23	equals/hashCode contract.	Equal objects must have same hashCode. Unequal objects MAY share hashCode
24	Checked vs unchecked exceptions.	Checked = compiler enforces handling (IOException). Unchecked = RuntimeExce
25	Will finally execute if try returns?	Yes. Only skipped if JVM exits or thread killed.
26	Difference between abstract class and interface.	Abstract class can have state and constructors; one parent only. Interface = contra
27	Method overloading vs overriding.	Overloading: same name, different params, compile-time, one class. Overriding: s
28	Why streams over for loops?	Declarative, composable, parallelizable. Slower for tight numeric loops — use arra
29	Optional — when to use.	Return type when result may be absent. Forces caller to handle missing case. Don
30	What is a functional interface?	Interface with exactly ONE abstract method. @FunctionalInterface annotation enfo

Web Questions

#	Question	Answer hooks
31	Servlet lifecycle.	init (once), service → doGet/doPost (per request), destroy (once on shutdown).
32	forward vs sendRedirect.	forward: server-side, same request, URL unchanged. sendRedirect: HTTP 302, ne
33	== vs === in JS.	== coerces types (1 == "1" true). === doesn't (1 === "1" false). Always use === un
34	var vs let vs const.	var: function-scoped, hoisted. let: block-scoped, TDZ. const: block-scoped, no rea
35	How to center a div?	Flex: display:flex; justify-content:center; align-items:center; on parent.
36	Promise vs async/await.	async/await is syntactic sugar over Promises. Same async semantics, looks like s
37	event loop.	JS is single-threaded. Async callbacks queue on task/microtask queues. Event loo
38	CSS specificity order.	(inline, IDs, classes, types). Higher tuple wins. !important overrides everything (av
39	JSP vs HTL.	HTL: secure-by-default, no logic in template, designer-readable, faster. JSP: legac
40	HTTP GET vs POST.	GET: idempotent, params in URL, cacheable. POST: state-changing, body, not ca

15-Minute Demo Walkthrough Script

Practice this end-to-end. The flow shows architecture understanding, not just clicks. Time yourself — aim for 12-15 minutes.

Minute	What to show	What to say
0-1	Open /system/console/bundles	"This is OSGi — every AEM feature is a bundle. My code bundle is com.genc.core
1-2	IDE — pom.xml hierarchy	"Multi-module Maven project. core has Java code. ui.apps has components and cl
2-4	CRXDE — show /apps/genc-aem-training/components	"Three components. Course Cards in the Content group; Header and Foot
4-6	Open coursecards.html + CourseListModel.java side-by-side	"HTL uses data-sly-use to instantiate the Sling Model. The model adapts from Req
6-7	/system/console/configMgr — find CourseConfigService	"OSGi configuration: DAM root path and default limit. These are environment-spec
7-9	Open /assets.html → courses folder	"12 Content Fragments based on the Course model. Each has title, description, da
9-10	/aem/tags.html → genc-aem_training namespace	"Tag namespace with 10 course categories. The CF Model's courseCategory field
10-11	Sites Console → home page → edit	"4 pages all using the Content Page editable template. Header and Footer come fr
11-12	/conf/genc-aem-training/settings/wcm/templates/content-page	"Editable template structure section locks header and footer (XF placeholders). L
12-14	/security/users.html → show 3 groups + ACLs	"RBAC: authors group has full rights but DENY on /content/experience-fragments.
14-15	Open issues log section	"Documented 10 bugs hit and fixed. Top one: composite multifield needed _cq_ten

■ **Note:** Don't read this verbatim. Internalize the narrative arc: **structure** → **code** → **data** → **authoring** → **security** → **lessons learned**. If asked a question mid-demo, answer briefly and steer back.

Appendix A — Quick-Reference URLs

Memorize these. Evaluators ask *'where would you go to check X?'* all the time.

AEM Author Console URLs

URL	What it does
/sites.html	Sites Console — browse and edit pages
/assets.html	Assets Console — DAM, CFs, images
/aem/tags.html	Tag administration
/aem/inbox	Workflow inbox
/projects.html	Project workspace
/aem/start.html	AEM home / dashboard
/security/users.html	User and group administration
/etc/replication.html	Replication agents
/tools.html	Tools console (templates, workflows, etc.)
/conf/wcm/templates	Templates by configuration

Developer / Debug URLs

URL	What it does
/system/console	OSGi Web Console home
/system/console/bundles	Bundle list — check ACTIVE state
/system/console/components	OSGi components — check status
/system/console/configMgr	OSGi configurations
/system/console/status-slingmodels	Registered Sling Models
/system/console/servletresolver	Test which servlet handles a URL
/system/console/jcr-resolver	Test Sling resource resolution
/system/console/services	OSGi service registry
/system/console/healthcheck	Health checks
/crx/de/index.jsp	CRXDE Lite — JCR browser
/crx/packmgr	Package Manager
/crx/explorer	CRX Explorer (legacy)

Your project URLs (Author 4502)

URL	What it shows
/sites.html/content/genc-aem-training/us/en/project	Your 4 pages
/aem/tags.html/content/cq:tags/genc-aem_training	Your tag namespace
/assets.html/content/dam/genc-aem-training/courses	Your 12 Content Fragments
/editor.html/conf/genc-aem-training/settings/wcm/templates/content-page-structure-template	Your Content Page template (editable)
/editor.html/content/experience-fragments/genc-aem-training/site/header/master.html	Header XF
/editor.html/content/experience-fragments/genc-aem-training/site/footer/master.html	Footer XF

Appendix B — Day-Before Cheat Sheet

If you only review one page the night before, make it this one.

Definitions in 10 seconds

- **AEM** — Adobe's enterprise WCM on Java + Sling + OSGi + JCR.
- **JCR** — hierarchical content repository; everything is a node with properties.
- **Sling** — REST-ful framework mapping URLs to JCR resources.
- **OSGi** — modular Java runtime; every AEM feature is a bundle.
- **HTL** — server-side template language; secure by default; replaces JSP.
- **Sling Model** — Java class with @Model that adapts a request/resource into typed Java.
- **Component** — folder under /apps with HTL + dialog + content.xml.
- **Editable Template** — author-managed page structure + policies, stored under /conf.
- **Content Fragment** — structured headless data, in DAM.
- **Experience Fragment** — authored UI block for reuse across pages/channels.
- **Tag** — hierarchical taxonomy node under /content/cq:tags.
- **Replication** — copying content author → publisher.
- **Dispatcher** — Apache module: caches HTML, load-balances, filters URLs.
- **MSM** — multi-site management via blueprint + live copies.

Sling Model in 8 lines

```
@Model(adaptables = {SlingHttpServletRequest.class, Resource.class},
       defaultInjectionStrategy = DefaultInjectionStrategy.OPTIONAL)
public class MyModel {
    @ValueMapValue private String headingText;
    @OSGiService    private MyService service;
    @PostConstruct void init() { /* logic */ }
    public String getHeadingText() { return headingText; }
}
```

OSGi service in 8 lines

```
@Component(service = MyService.class)
@Designate(ocd = MyConfig.class)
public class MyServiceImpl implements MyService {
    private String someValue;
    @Activate void activate(MyConfig cfg) { someValue = cfg.someValue(); }
    @Override public String getSomeValue() { return someValue; }
}
```

Servlet by resource type in 8 lines

```
@Component(service = Servlet.class,
    property = {
        "sling.servlet.resourceTypes=genc/components/api",
        "sling.servlet.methods=GET",
        "sling.servlet.extensions=json"
    })
public class MyServlet extends SlingSafeMethodsServlet {
    @Override protected void doGet(...) { /* write JSON */ }
}
```

HTL pattern with composite multifield

```
<nav data-sly-use.items="navLinks"
     data-sly-list.link="${items.children}">
    <a href="${link.linkURL @ context='uri'}.html">${link.linkText}</a>
</nav>
```

The bugs you fixed (one-liners)

- filter.xml: `<workspaceFilter>` root, not `<__workspaceFilter__>`.
- /conf belongs in ui.content's filter.xml, not ui.apps'.
- Composite multifield items are child nodes; HTL needs data-sly-use + .children.
- First-drop empty multifield crashes HTL — add _cq_template.xml with initial item.
- Format dates in Java; don't pass Calendar to HTL.
- TagManager can return null — extract last path segment as fallback.
- Namespace hyphens auto-convert to underscores in JCR node names.
- crx:replicate is separate from jcr:all — grant explicitly.

Mental check before the eval

- Can I explain Sling resource resolution in 30 seconds? (URL → path → resourceType → script)
- Can I draw the author/publisher/dispatcher diagram from memory?
- Do I know the difference between CF and XF cold?
- Can I write a Sling Model from scratch on paper?
- Can I write a Sling Servlet by resource type from scratch?
- Do I have ONE good bug story ready to tell?
- Have I rehearsed my 15-minute demo at least twice?
- Can I answer all 40 mock interview questions out loud without notes?

Good luck. You built a real project end-to-end, hit real bugs, and fixed them. That's more than most candidates can claim. Walk in calm.